

COSC264 · TERM 4

## Module 5

# Transport Layer Protocols

---

TCP and UDP

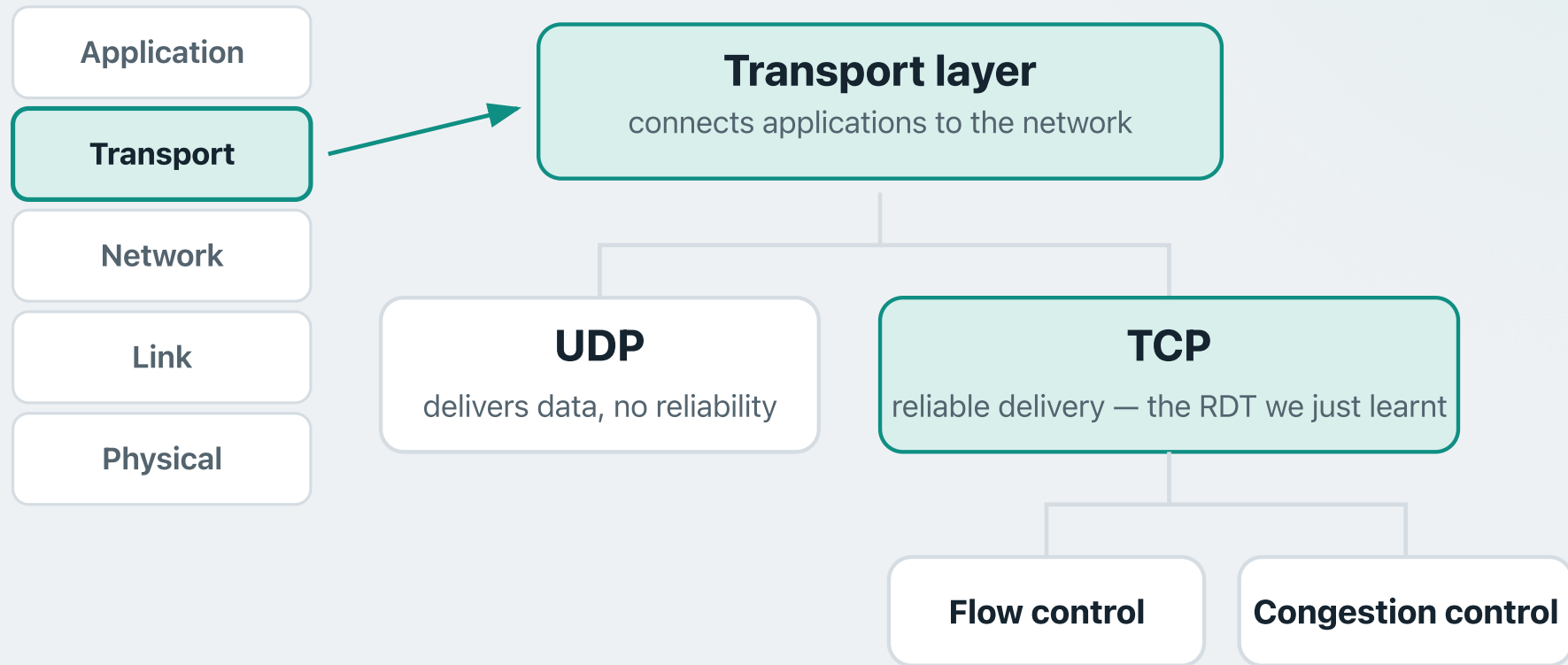
[cosc264.up.railway.app](https://cosc264.up.railway.app)

# Module 4 — what we covered

- Reliable Data Transfer (RDT) moves data reliably over an unreliable channel.
- From rdt 1.0 to rdt 3.0 we used checksums, acknowledgements (ACKs), sequence numbers, and a timer to detect errors and loss.
- RDT up to rdt 3.0 is **stop-and-wait**, which is inefficient, so we added **pipelining** — several packets in flight at once, capped by a **sender window**.
  - **Go-back-N** uses a sender window and **cumulative ACKs**; the receiver discards anything that arrives out of order.
  - **Selective repeat** improves on that with a **receiver buffer**, so out-of-order packets are kept rather than thrown away.

# Module 5 roadmap

Recall: the Internet stack has five layers



# Transport layer provides an end-to-end channel between applications

Suppose a network application runs process A and process B on separate devices and needs an end-to-end communication channel.

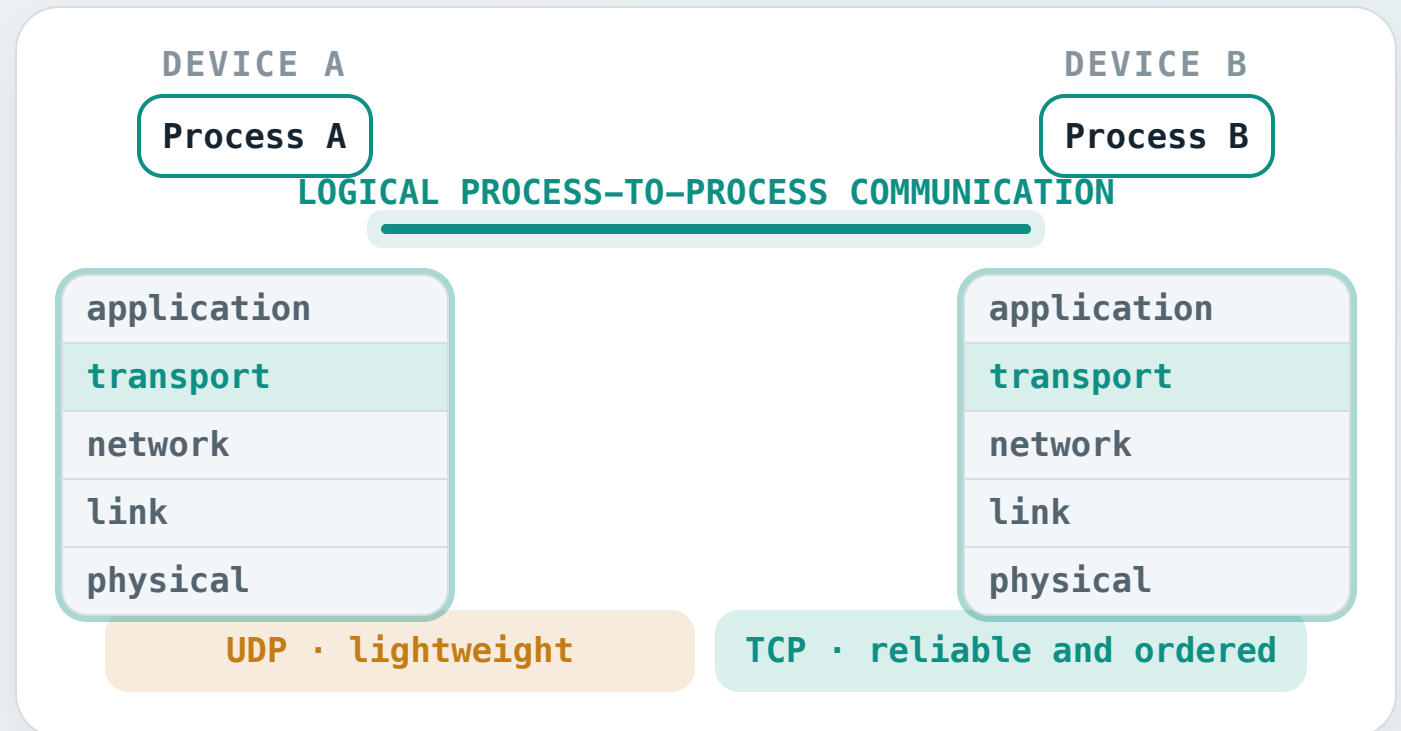
**Transport layer provides that channel.** It provides logical end-to-end communication between two processes.

**Abstraction.** The transport layer abstracts away the network's lower-layer mechanics, keeping them invisible to applications.

**Service choice.** Applications can choose UDP or TCP for different delivery needs.

UDP

TCP



# Mechanisms utilized by transport-layer protocols

To provide this end-to-end abstraction, transport protocols use several mechanisms.

## Error detection

A checksum helps detect corruption in a received transport unit.

TCP + UDP

## Multiplexing and demultiplexing

Next up.

TCP + UDP

## Reliable data transfer (RDT)

Module 4.

TCP

## Flow and congestion control

Later in this module.

TCP

# Processes reach the network through sockets

A single host can run many network applications over multiple processes. Each process may need to send and receive traffic through the network.

A process does not talk to the network directly. It sends and receives through a **socket**.

A socket is the transport layer's endpoint — the door a process writes its outgoing data into, and reads its incoming data out of.

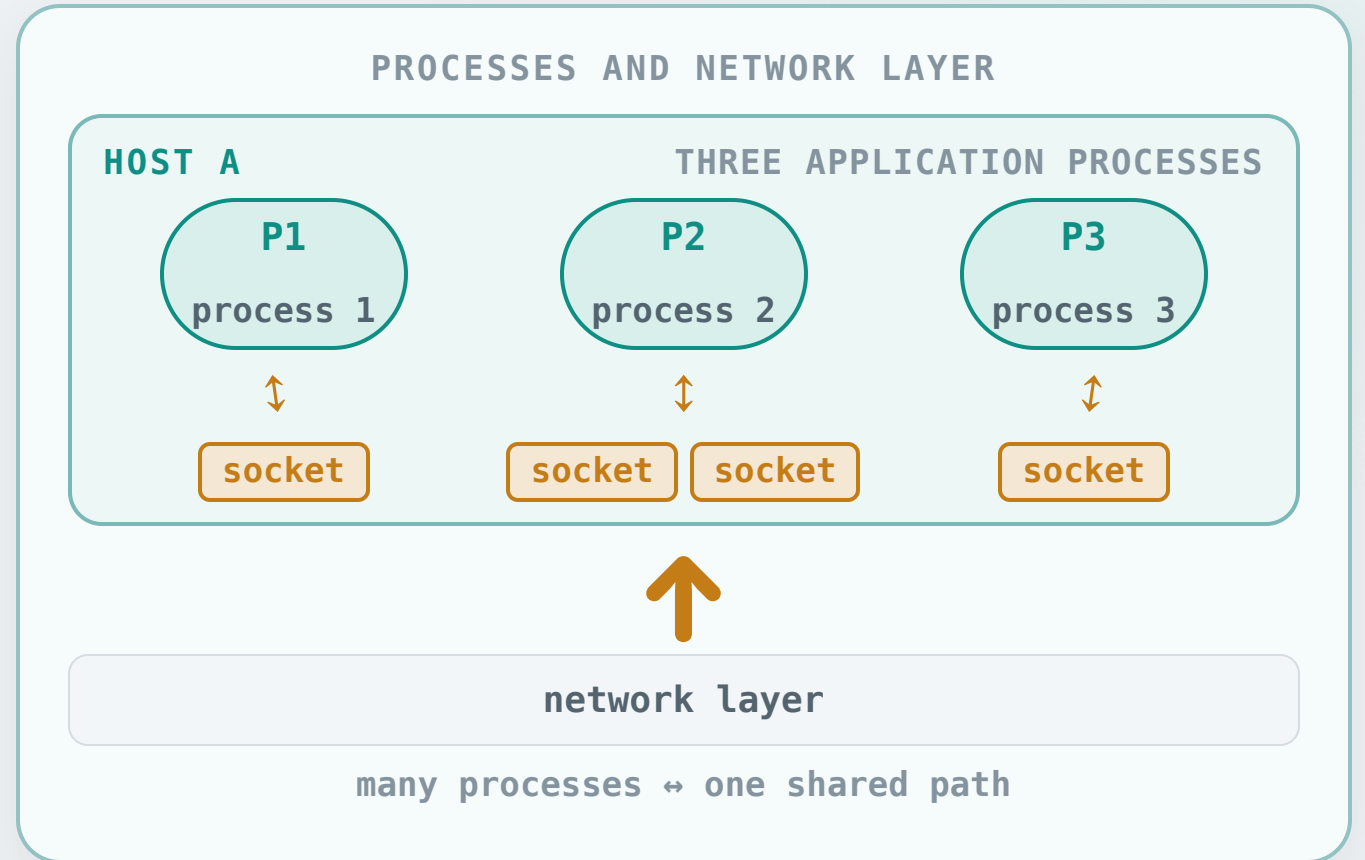
A process may hold **more than one socket**.



# Why multiplexing is needed

The network layer — **IP** — only ever looks at **IP addresses**. To it a host is a single object at a single address; it has no view of the sockets inside.

So when a packet arrives, IP can deliver it to the **host** — but it cannot tell which **socket** the data inside should go to.

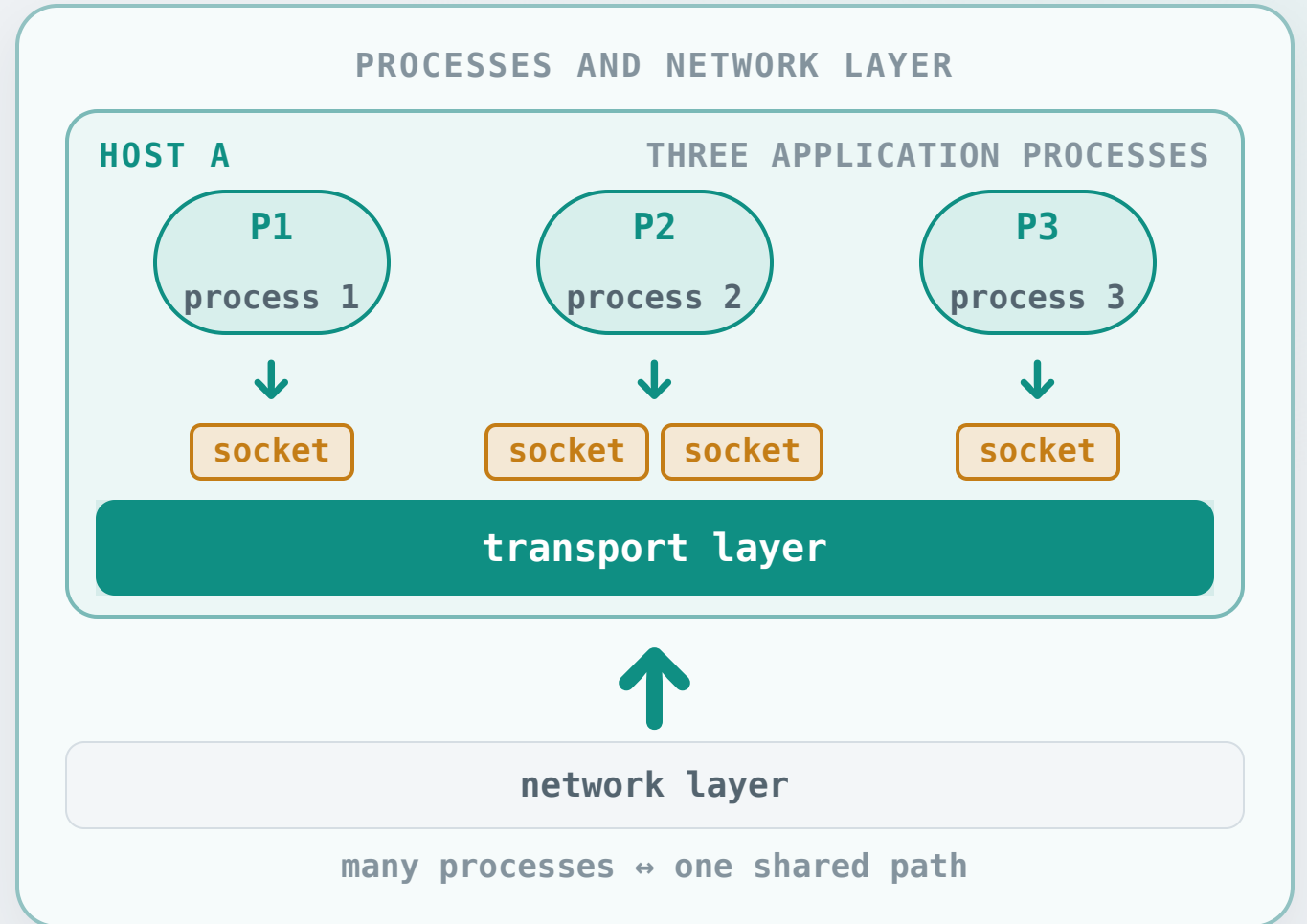


# Why multiplexing is needed

The network layer — **IP** — only ever looks at **IP addresses**. To it a host is a single object at a single address; it has no view of the sockets inside.

So when a packet arrives, IP can deliver it to the **host** — but it cannot tell which **socket** the data inside should go to.

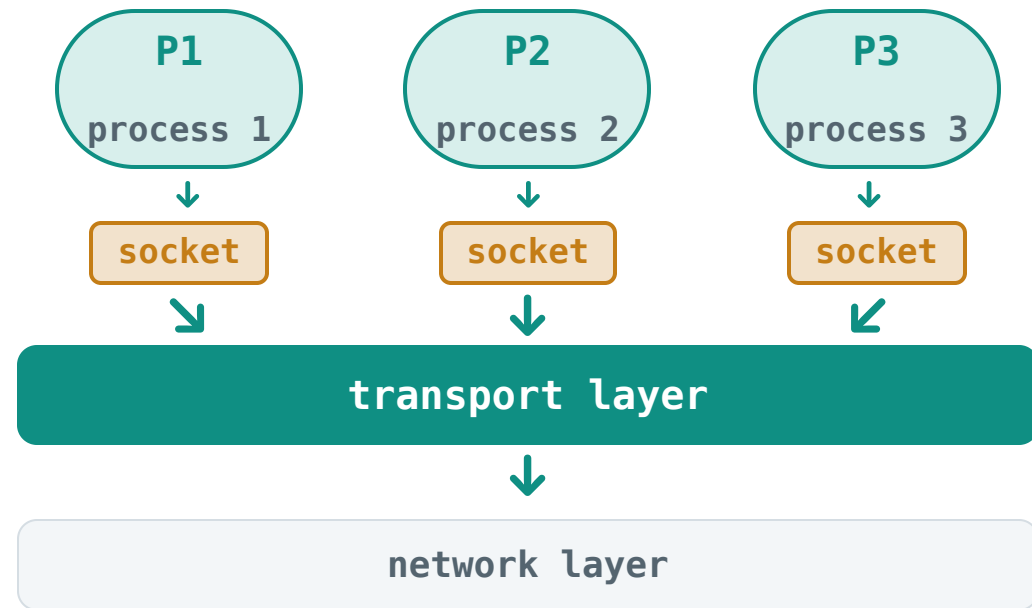
The transport layer coordinates this traffic in both directions so each socket gets exactly its own data. This is known as **multiplexing/demultiplexing**.



# Multiplexing sends; demultiplexing delivers

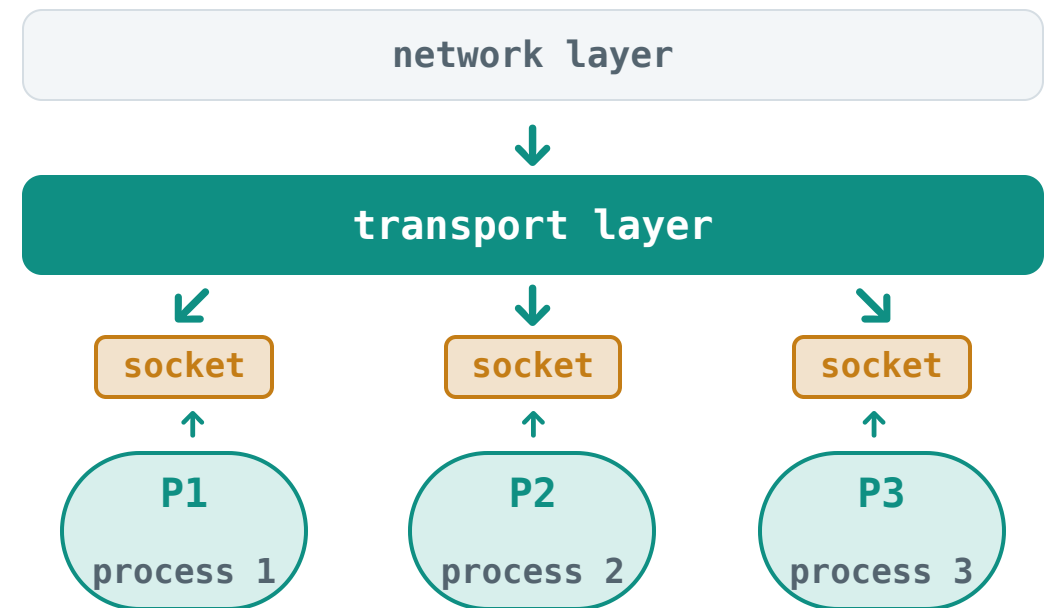
## Multiplexing

The transport layer gathers data from the socket of every application process and passes it down to the network layer.



## Demultiplexing

The transport layer receives data from the network layer and delivers each piece to the correct socket.



# UDP demultiplexing

Suppose the network layer uses the destination IP address to deliver the following IP packet to some host.

**How does UDP perform demultiplexing afterwards?**

**UDP looks at two fields.** UDP uses the destination IP address in the IP header and the destination port in the UDP header to identify the correct socket.

For this packet, the identified socket is the one bound to **192.0.2.80 : 19157**.

## ONE IP PACKET CARRYING ONE UDP DATAGRAM

### IP HEADER

SOURCE IP  
**198.51.100.23**

DESTINATION IP  
**192.0.2.80**

### UDP HEADER

SOURCE PORT  
**42345**

DESTINATION PORT  
**19157**

application data

↓ UDP socket · **192.0.2.80:19157**

# TCP demultiplexing

Suppose the network layer uses the destination IP address to deliver the following IP packet to some host.

## How does TCP perform demultiplexing afterwards?

**TCP looks at four fields.** TCP uses the source IP address and destination IP address in the IP header, plus the source port and destination port in the TCP header, to identify the correct connected socket.

For this packet, the identified socket is the one bound to the pair **203.0.113.10 : 46428 ↔ 192.0.2.80 : 80**.

### TCP CONNECTION IDENTITY

#### Source endpoint

SOURCE IP ADDRESS

**203.0.113.10**

SOURCE PORT

**46428**

#### Destination endpoint

DESTINATION IP ADDRESS

**192.0.2.80**

DESTINATION PORT

**80**

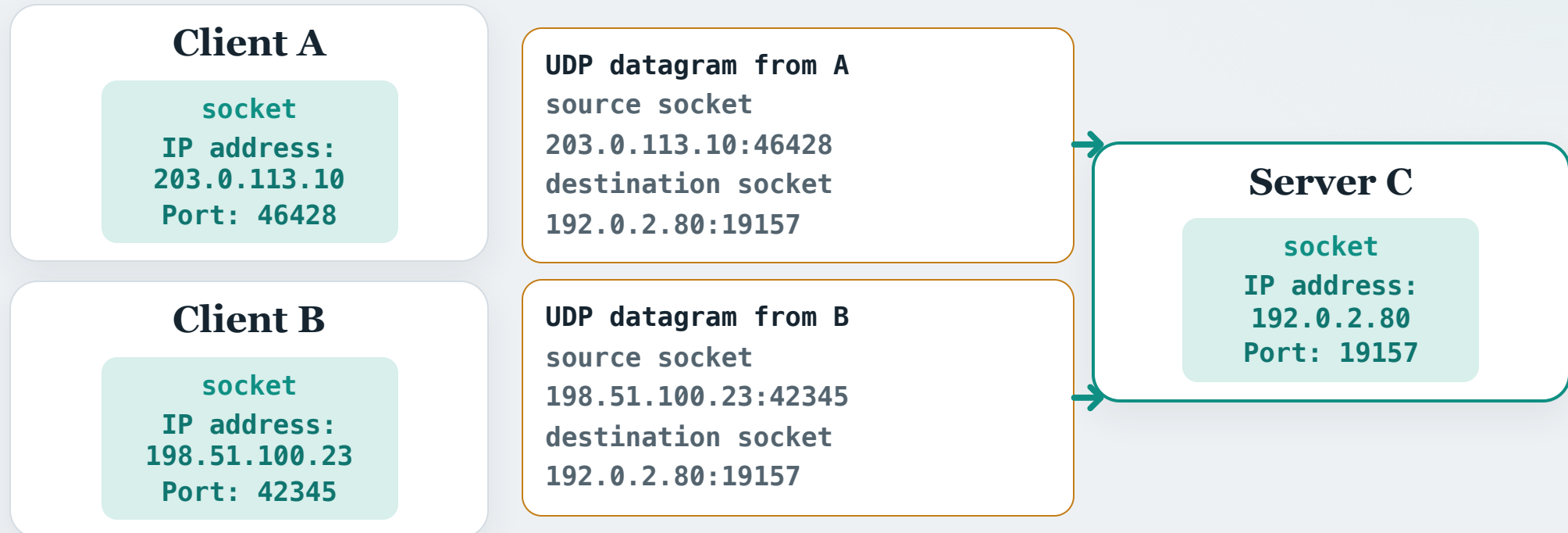
**203.0.113.10:46428 ↔ 192.0.2.80:80**

# UDP is unreliable and connectionless

- **Unreliable.** UDP does not guarantee delivery, ordering, or protection against duplicates.
- **Connectionless.** A sender does not set up a connection before sending.
- Why not? Because UDP identifies the socket by the **destination alone**. Put a destination IP address and port in the header, and UDP hands the datagram to that socket.
- It does not care where the datagram came from, so there is **no binding** between a particular sender and a particular receiver.

# Different sources, one socket

- Since UDP is connectionless, two different senders can send to the **same destination socket**, as depicted in this diagram.
- **But then how does the destination reply to each of them?**
- Every datagram also carries its **source IP address and port** — the return address the destination replies to.



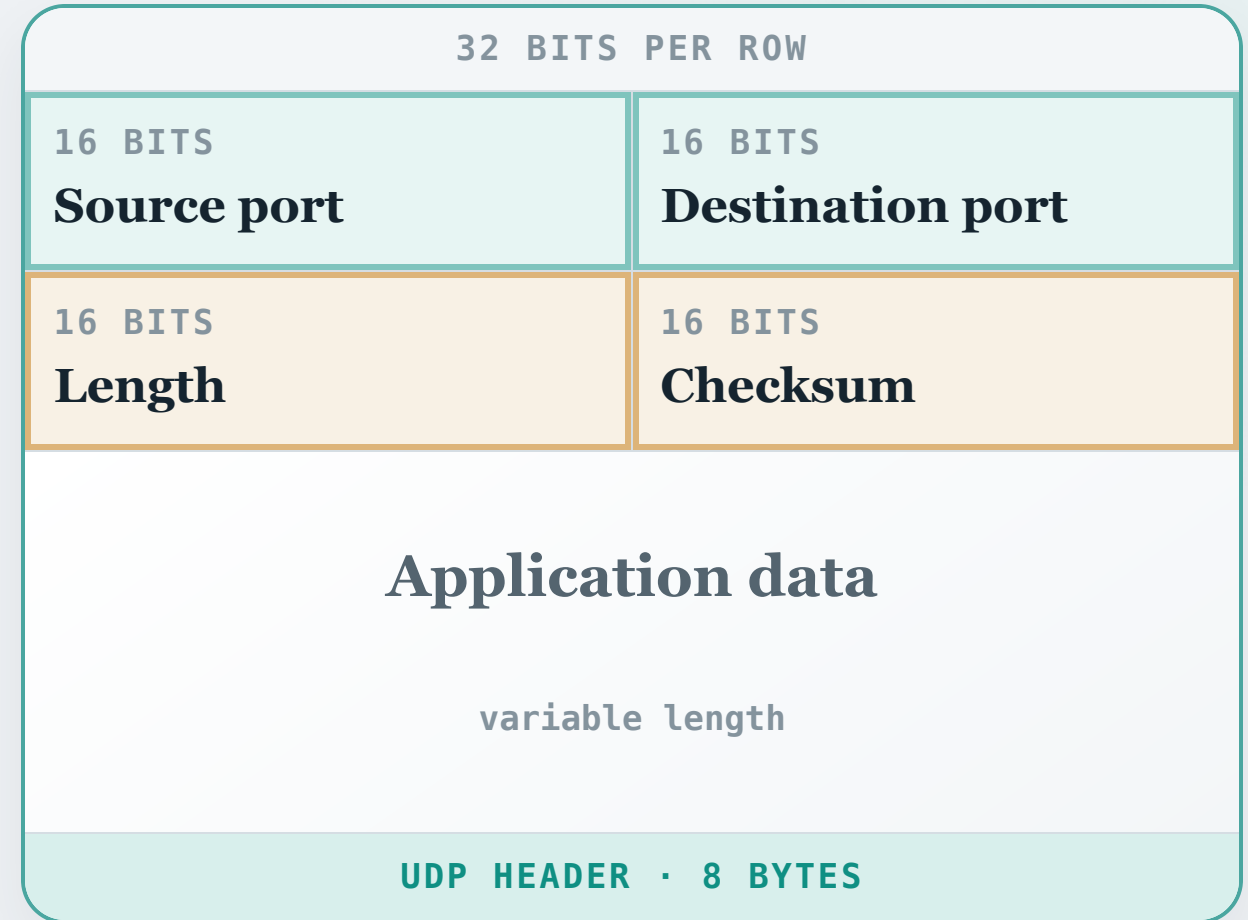
# Breaking down the UDP header

**Always 8 bytes.** Four 16-bit fields and nothing optional, so a UDP header is the same size on every datagram.

**Ports.** The 16-bit source and destination ports identify the sending and receiving application endpoints.

**Length.** The total number of bytes in the UDP header and application data; the minimum is eight.

**Checksum.** Covers the UDP header and the entire payload data to detect corruption.



# Why do some applications choose UDP?

1

## No setup delay

An application can send immediately because UDP does not establish a transport connection first.

2

## Minimal connection state

Since UDP is connectionless, it keeps no per-connection state — no buffers, sequence numbers or windows.

3

## Small header

Every UDP message adds only an eight-byte transport header.

4

## Application control

UDP provides no reliability, so the application decides when to send and builds whatever recovery it needs — rather than taking TCP's fixed mechanism.

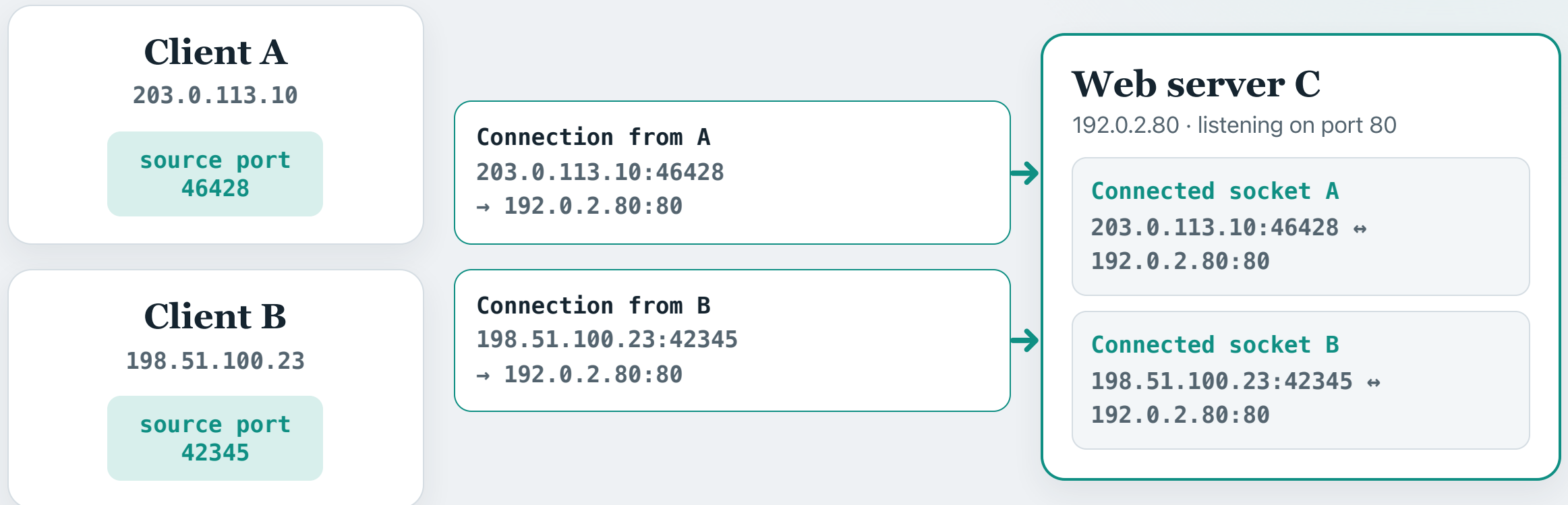
Common UDP examples include DNS lookups, real-time media, and online games.

# TCP is reliable and connection-oriented

- **Reliable.** TCP delivers the byte stream without loss, duplication or reordering — it has an RDT mechanism built in.
- **Connection-oriented.** TCP identifies a connection by **four** values: source IP address and port, destination IP address and port.
- So a sender must first run a **handshake** to set the connection up before it may send any data, and that connection ties one particular sender to one particular receiver.

# One connected socket per client

- If several clients want to talk to the same server, they each get their own **connection** — and so their own **connected socket** on that server.
- Compare with **UDP**: being connectionless, it lets many clients share a single socket.



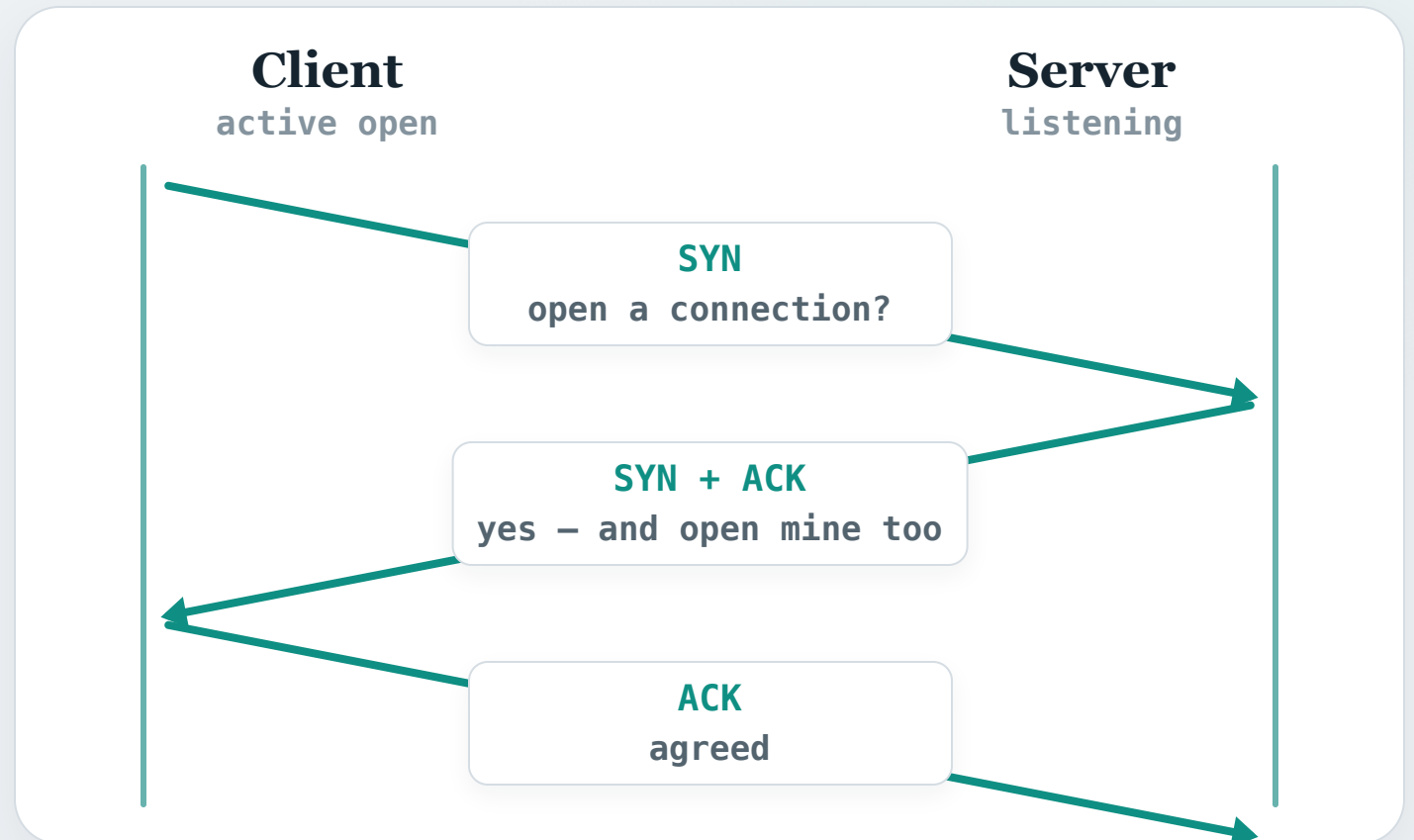
# TCP establishes a connection with a three-way handshake

**1 • SYN.** The client sends a segment with the **SYN** flag set, asking to open a conversation.

**2 • SYN + ACK.** The server agrees: in one segment it sets its own **SYN** flag and an **ACK** for the client's request.

**3 • ACK.** The client acknowledges the server's half, and the connection is open.

Now that both sides have heard from each other, they can start sending packets to one another.



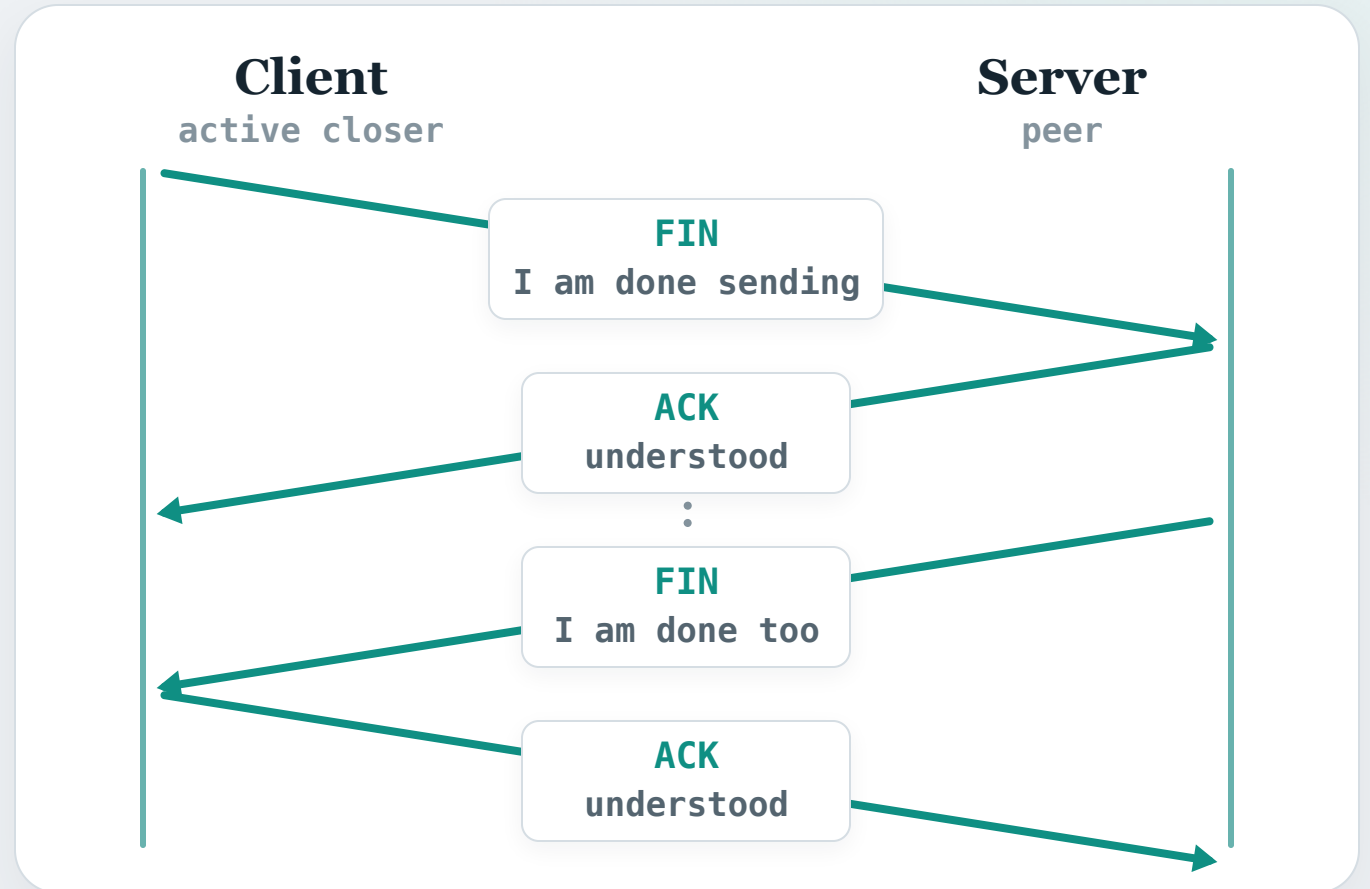
# How TCP closes a connection

**1 • FIN.** The client sets the **FIN** flag to say it has nothing more to send.

**2 • ACK.** The server acknowledges. But its own direction stays open, so it may still have things to send.

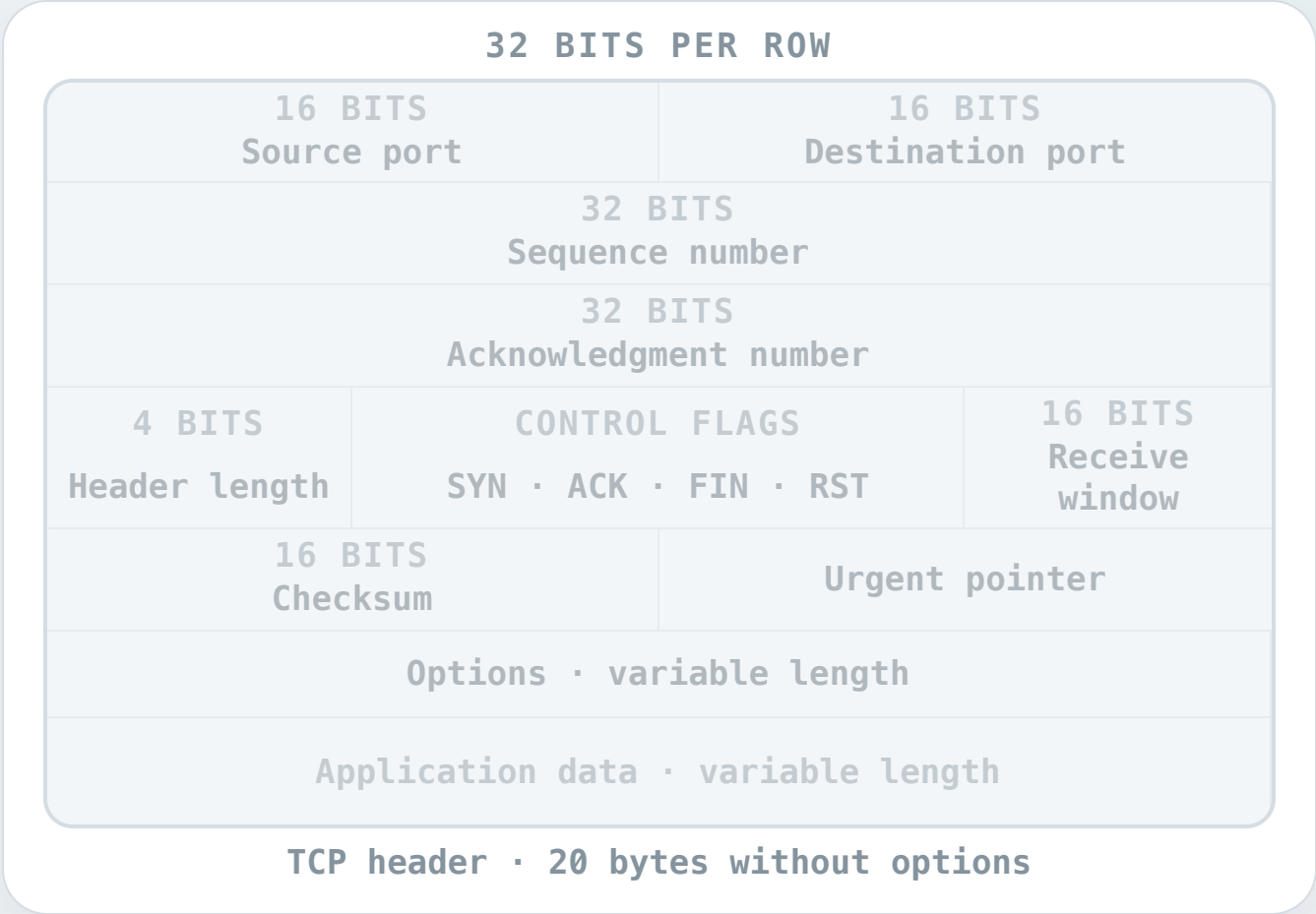
**3 • FIN.** When the server has finished too, it sets its own **FIN** flag.

**4 • ACK.** The client acknowledges, and the connection is closed.



# Key TCP header fields

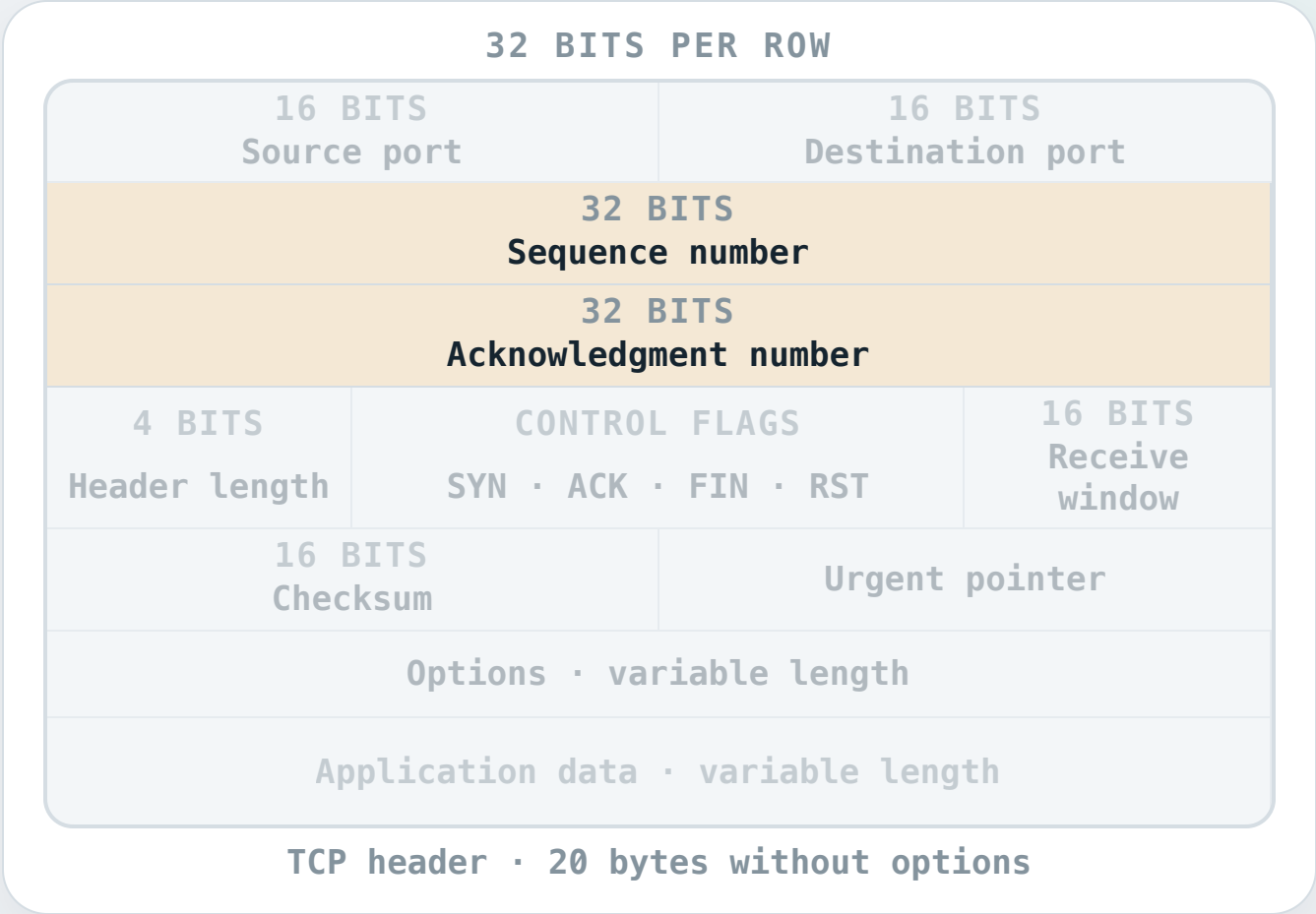
**20 bytes, or more.** Without options a TCP header is 20 bytes, and options can make it longer.



# Key TCP header fields

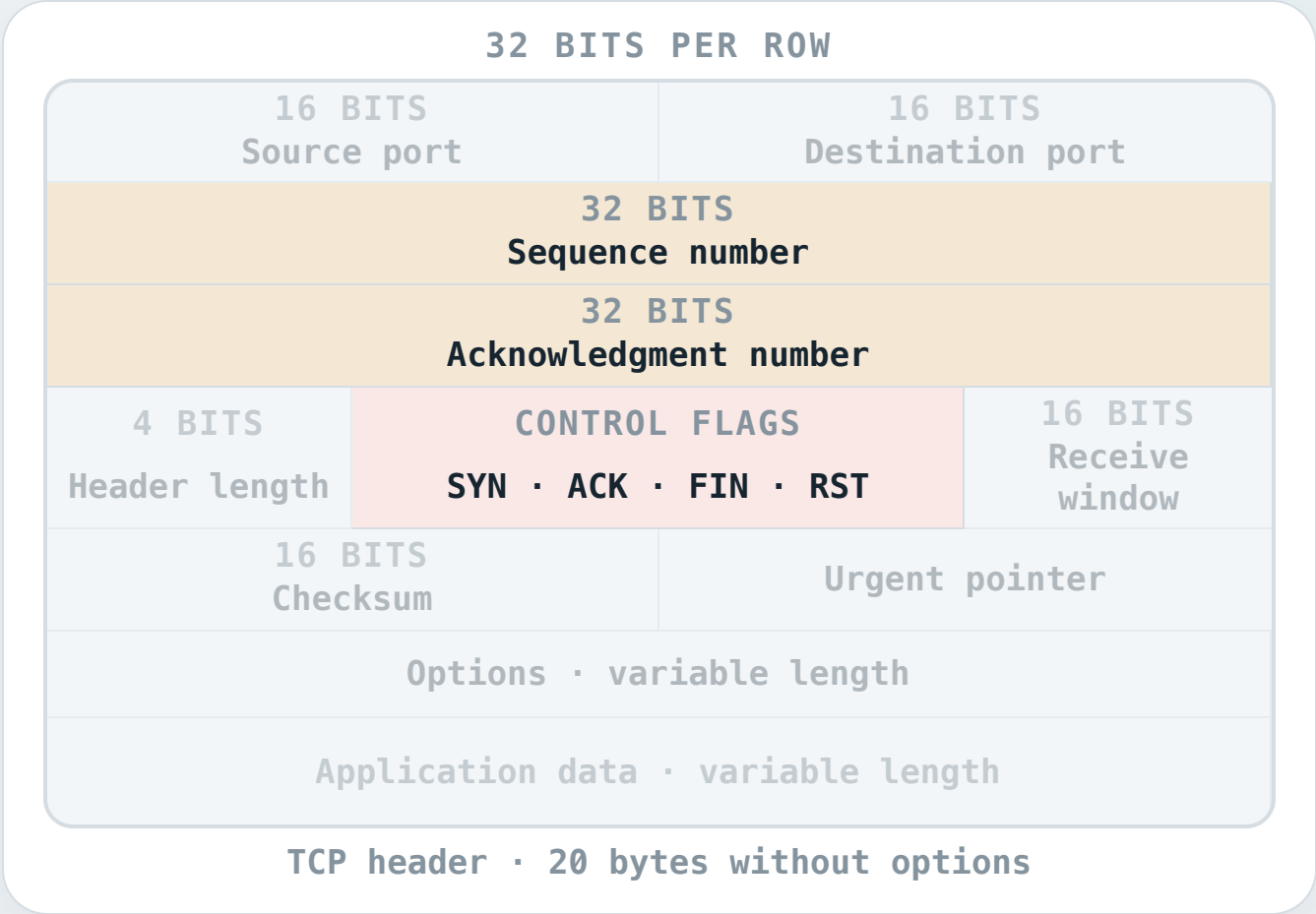
**Sequence and acknowledgment numbers.**

They identify the byte sent and the next byte expected. We come back to them next lecture.



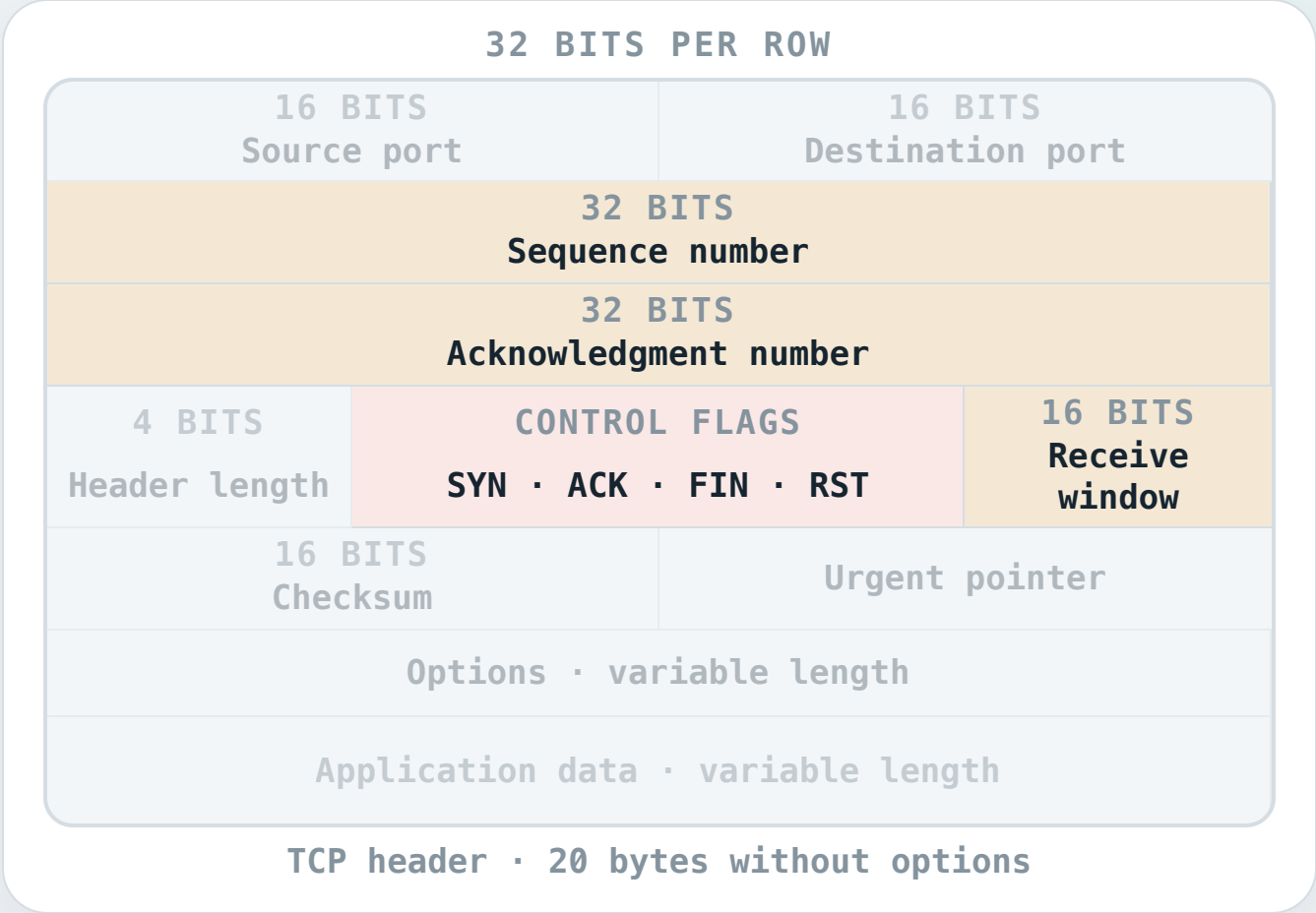
# Key TCP header fields

**Control flags.** Each is a one-bit field; a segment sets the relevant bit to 1 to signal SYN, ACK, FIN or RST.



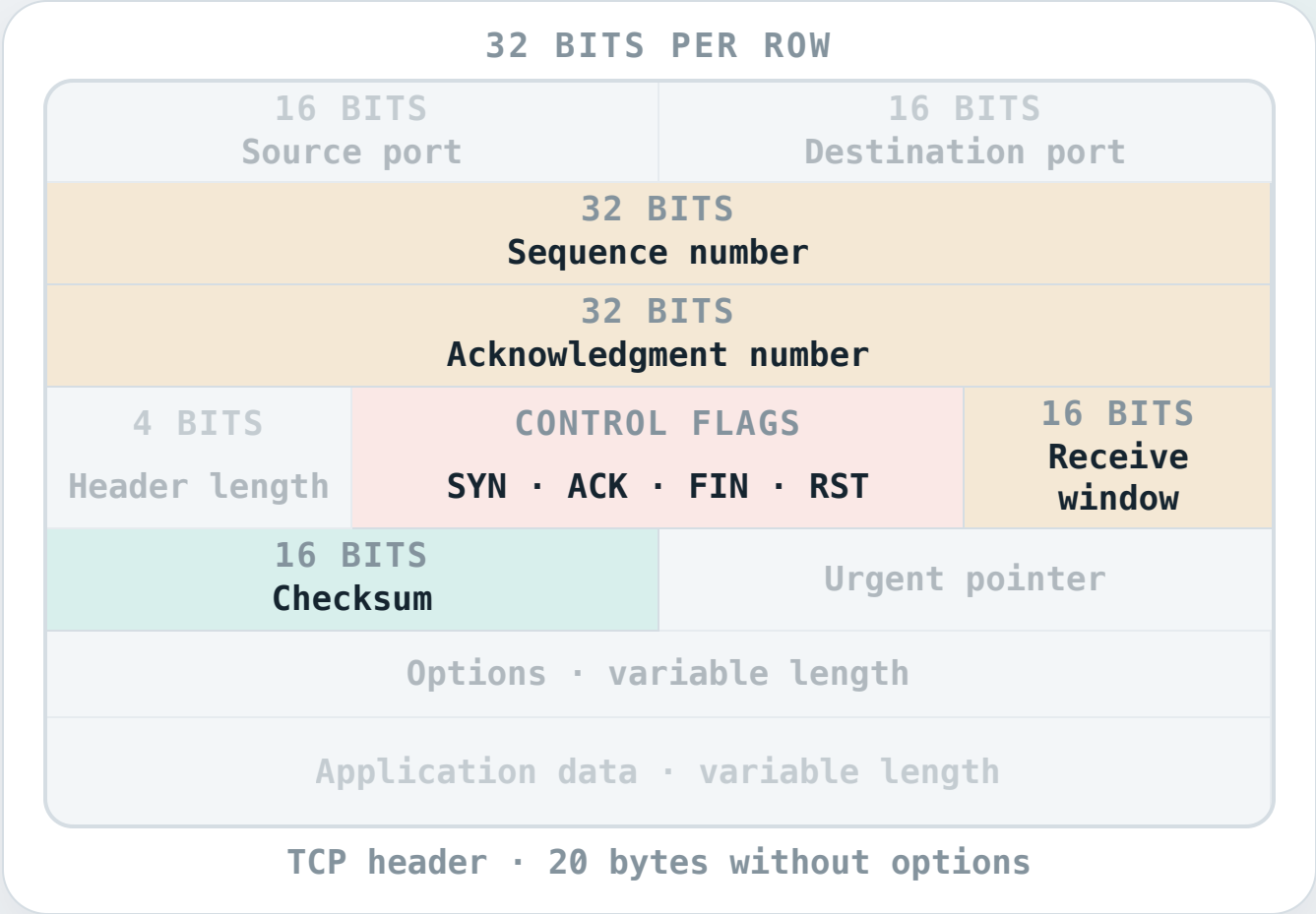
# Key TCP header fields

**Receive window.** We come back to this when we reach flow control.



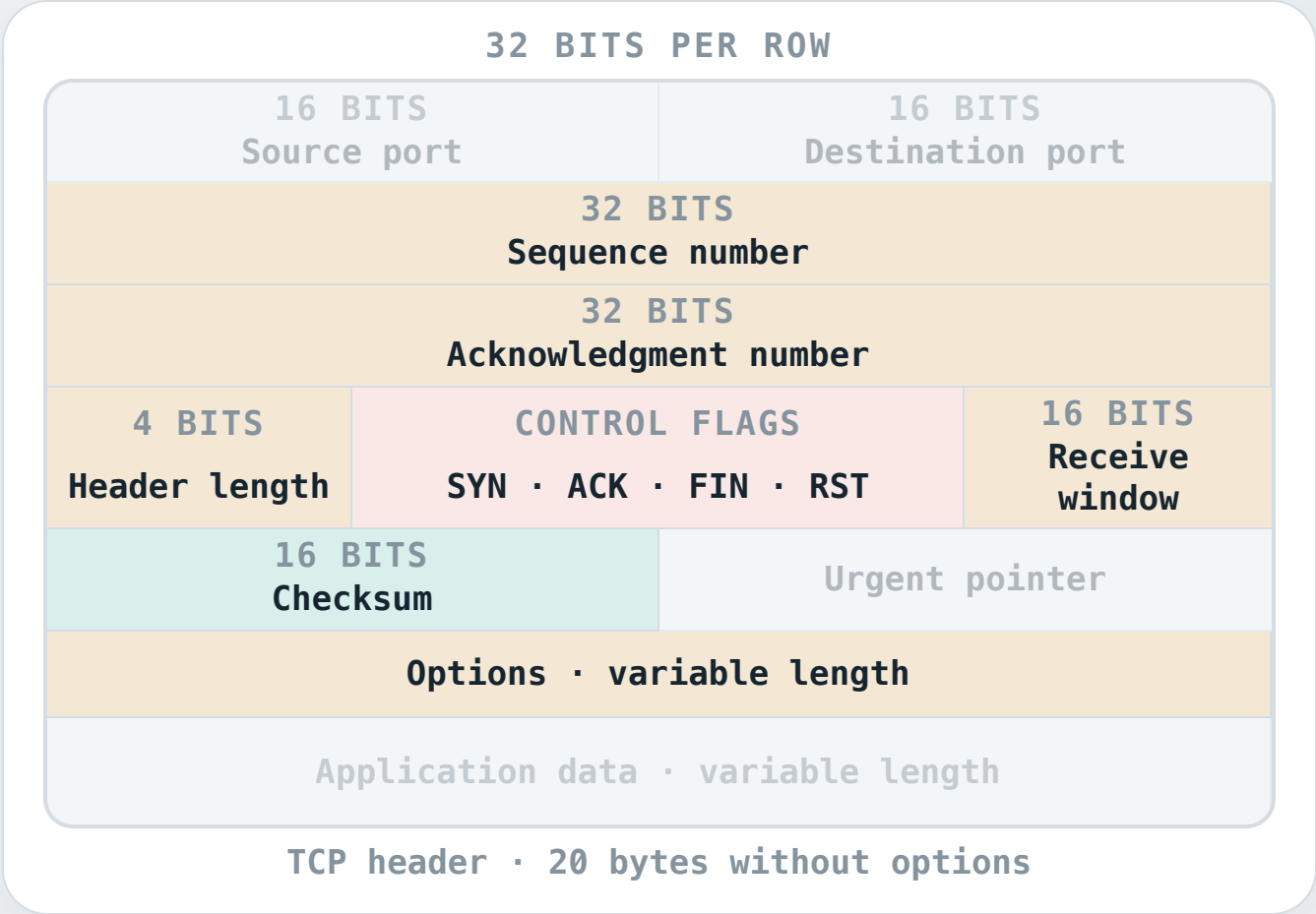
# Key TCP header fields

**Checksum.** It detects corruption in the TCP header and payload.



# Key TCP header fields

**Header length and options.** Header length shows where the data begins; options carry extra capabilities such as MSS.



# The RDT used by TCP — an overview

TCP uses a reliable data transfer protocol on top of IP to provide reliable, in-order byte delivery.

Its RDT is inspired by both **Go-Back-N** and **Selective Repeat** — the pipelined protocols we studied in Week 10.

## Pipelining and cumulative acknowledgments

The TCP sender uses a **window** of segments in flight, for **pipelining**.

The TCP receiver **keeps a buffer**, as in Selective Repeat, and acknowledges with **cumulative ACKs**, as in Go-Back-N.

But unlike Go-Back-N — where the ACK names the last in-order segment received — TCP's ACK names the **next byte expected**.

## Loss recovery

Retransmission happens when the timer times out or when duplicate ACKs arrive (**fast retransmit**).

# TCP sequence numbers count bytes

## Byte-level addressing

In the RDT protocols from Week 10, each sequence number advanced by one **packet** — between consecutive sender messages, the sequence number advanced by one.

In TCP, sequence numbers address individual **bytes** — so between consecutive sender segments, the sequence number advances by the **number of bytes** in the segment.

## Maximum Segment Size (MSS)

The largest payload a single TCP segment can carry — negotiated during the handshake. Typical value: **1460 bytes** on Ethernet (1500 MTU – 20 IP – 20 TCP).

## SYN and FIN consume a sequence number

SYN and FIN each consume exactly one sequence number, even though they carry no application data — this ensures they can be reliably acknowledged.

ISN = 1000 · MSS = 1000 BYTES · 3000 BYTES TO SEND

# Sequence and acknowledgment in action

The client wants to transmit 3000 bytes of application data over a TCP connection.

## The setup

With MSS = 1000 bytes, the 3000 bytes of data fill **three segments**.

Client

Data to be transmitted:



Server

ISN = 1000 · MSS = 1000 BYTES · 3000 BYTES TO SEND

# Sequence and acknowledgment in action

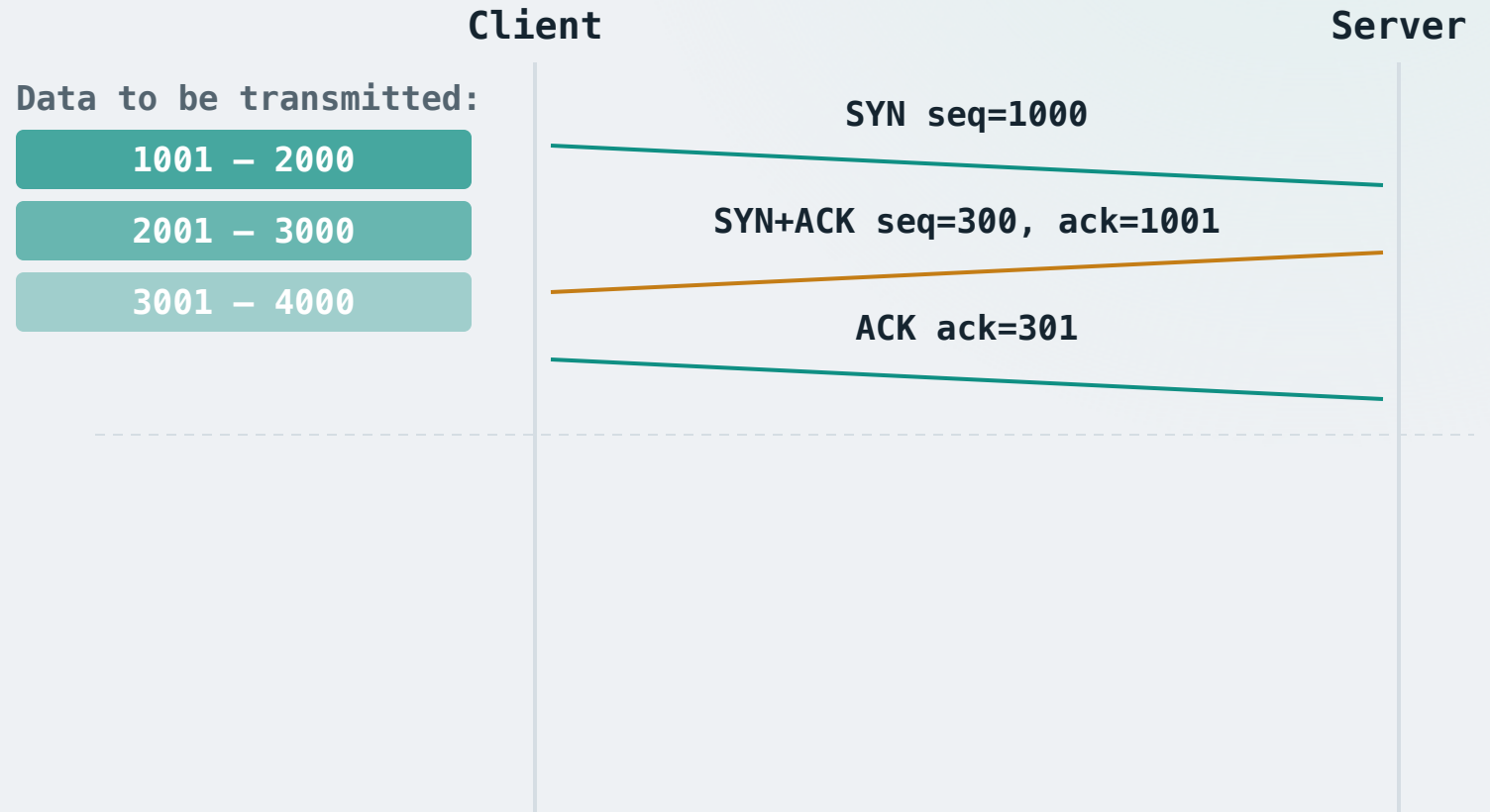
The client wants to transmit 3000 bytes of application data over a TCP connection.

## Three-way handshake

First, the client and server perform a three-way handshake to open the TCP connection.

Suppose ISN = 1000. Since SYN consumes one sequence number, the first data byte will have seq# **1001**.

The server's SYNACK carries **ACK = 1001**, which signals that it expects byte 1001 next.



ISN = 1000 · MSS = 1000 BYTES · 3000 BYTES TO SEND

# Sequence and acknowledgment in action

The client wants to transmit 3000 bytes of application data over a TCP connection.

## Transmit segment 1

The client sends the first 1000 bytes (seq = 1001).

Data to be transmitted:

1001 – 2000

2001 – 3000

3001 – 4000

Client

Server

SYN seq=1000

SYN+ACK seq=300, ack=1001

ACK ack=301

SEG 1 · seq=1001 · 1000 bytes

ISN = 1000 · MSS = 1000 BYTES · 3000 BYTES TO SEND

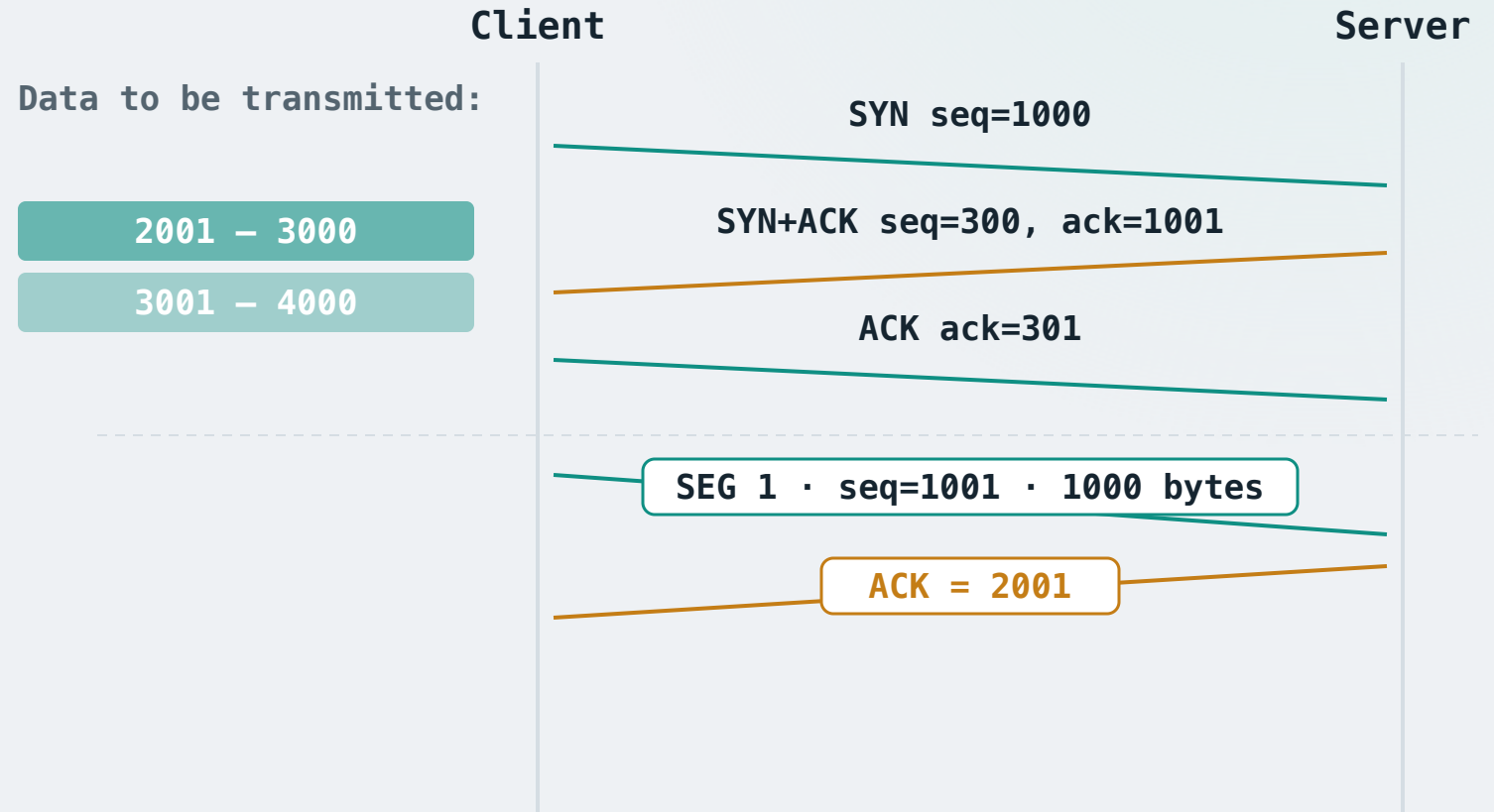
# Sequence and acknowledgment in action

The client wants to transmit 3000 bytes of application data over a TCP connection.

## Transmit segment 1

The client sends the first 1000 bytes (seq = 1001).

The server cumulatively acknowledges with **ACK = 2001** — it has every byte up to 2000 and expects byte 2001 next.



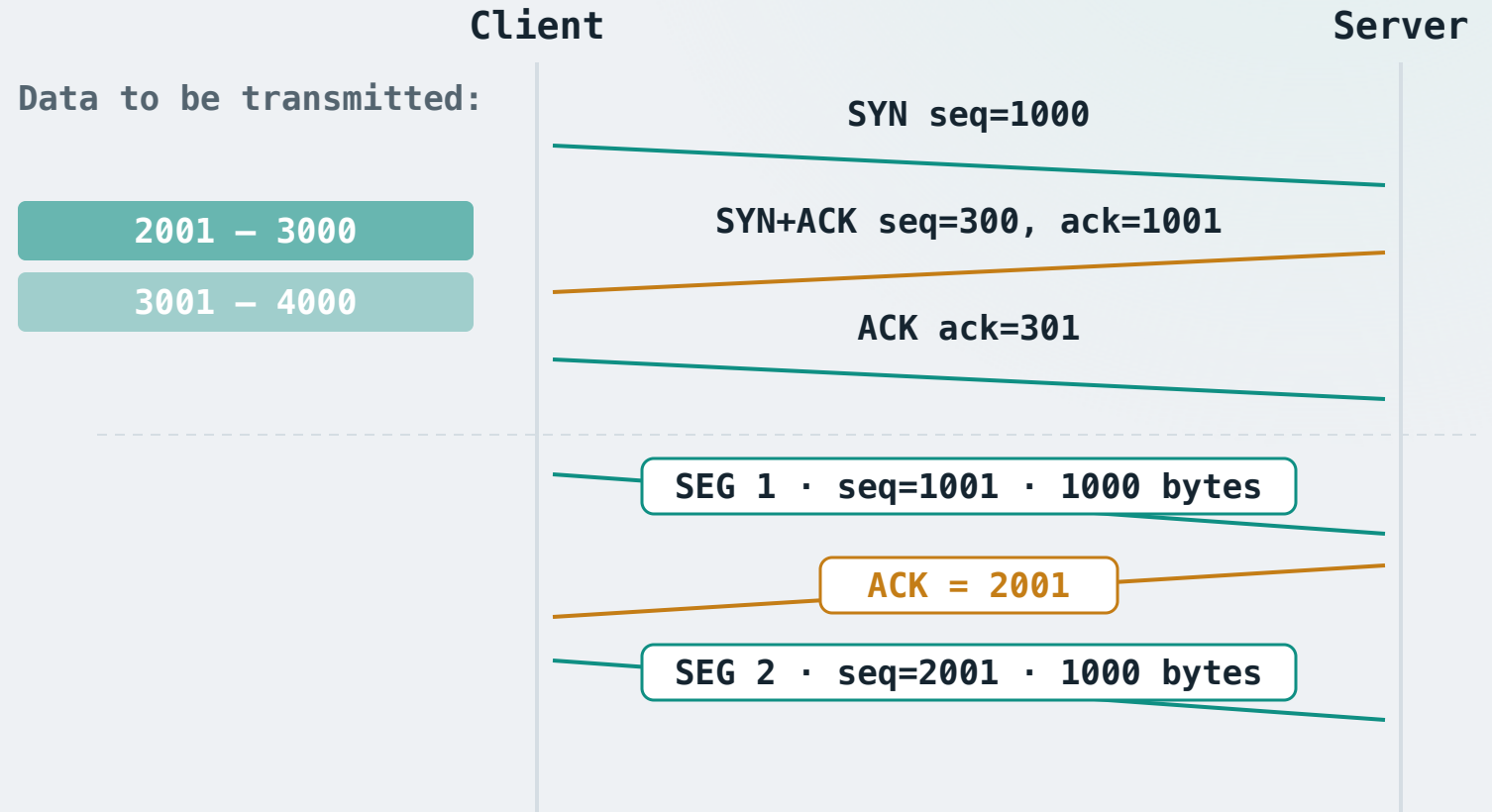
ISN = 1000 · MSS = 1000 BYTES · 3000 BYTES TO SEND

# Sequence and acknowledgment in action

The client wants to transmit 3000 bytes of application data over a TCP connection.

## Transmit segment 2

The client sends the next 1000 bytes (seq = 2001).



ISN = 1000 · MSS = 1000 BYTES · 3000 BYTES TO SEND

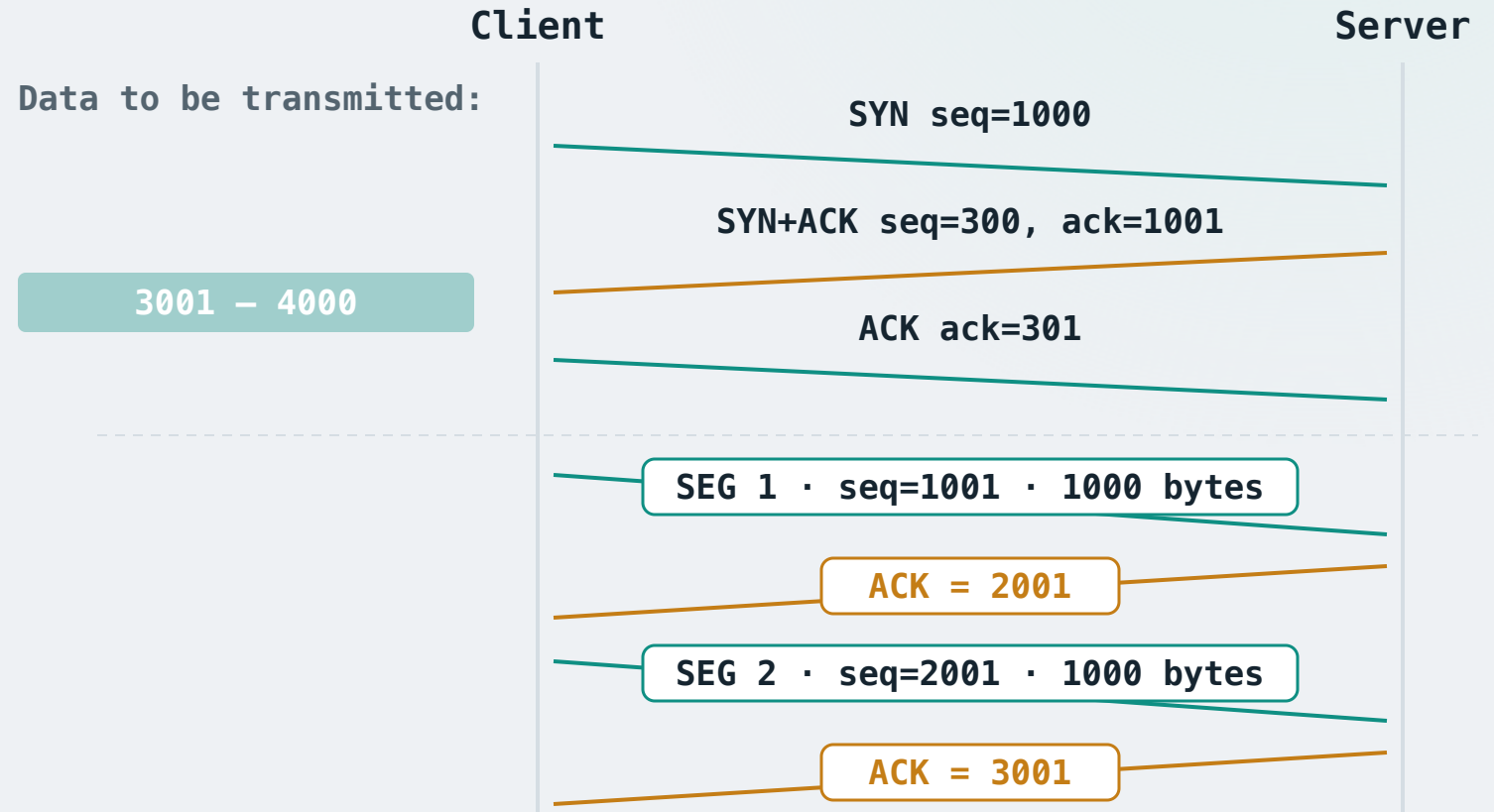
# Sequence and acknowledgment in action

The client wants to transmit 3000 bytes of application data over a TCP connection.

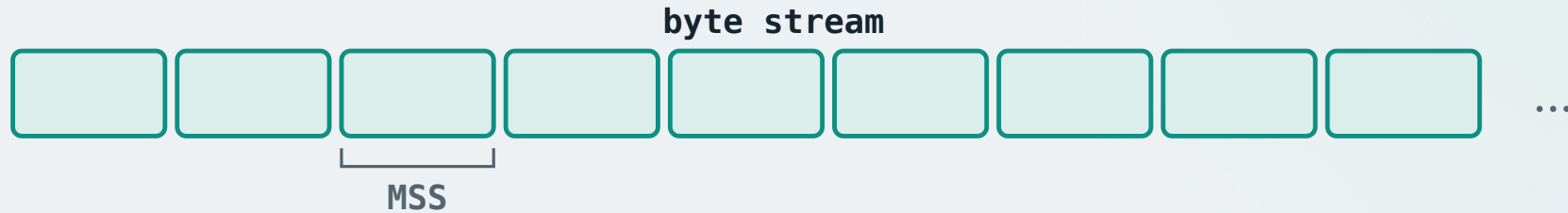
## Transmit segment 2

The client sends the next 1000 bytes (seq = 2001).

The server cumulatively acknowledges with **ACK = 3001** — confirming every byte up to 3000.



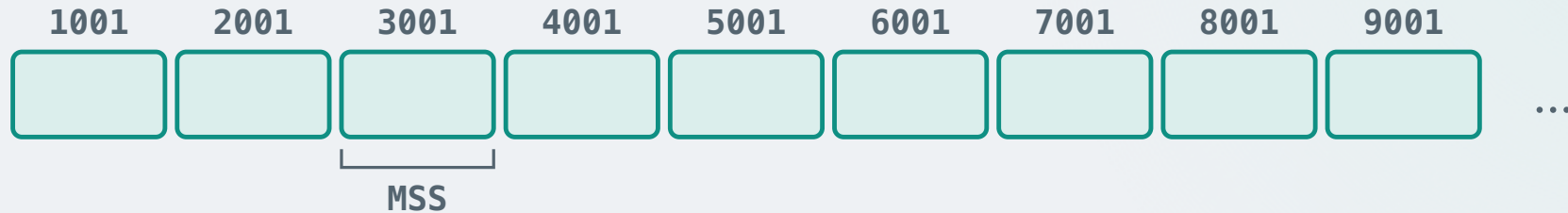
# The TCP sender rules: overview



## Data from application

Divide the byte stream into segments, typically **1 MSS** bytes each.

# The TCP sender rules: overview



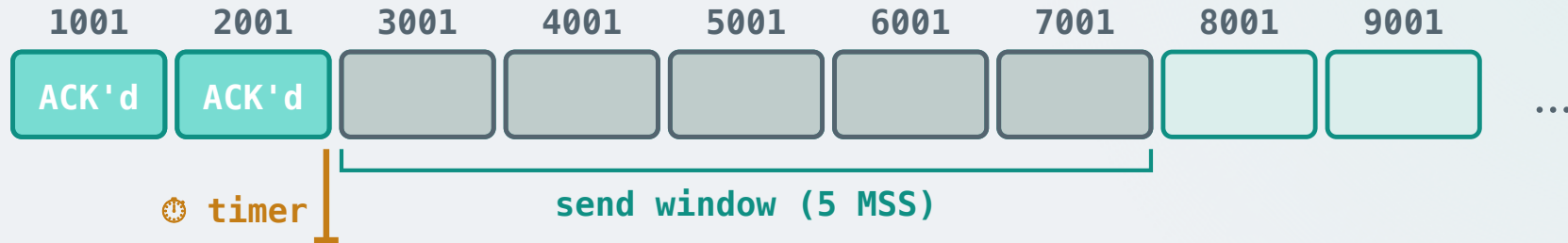
## Data from application

Divide the byte stream into segments, typically **1 MSS** bytes each.

Number each segment by its first byte, counting on from the **initial sequence number (ISN)** chosen in the handshake.

*e.g. ISN = 1000 and 1 MSS = 1000 bytes, so the segments start at 1001, 2001, 3001, ...*

# The TCP sender rules: overview



## Data from application

Divide the byte stream into segments, typically **1 MSS** bytes each.

Number each segment by its first byte, counting on from the **initial sequence number (ISN)** chosen in the handshake.

*e.g. ISN = 1000 and 1 MSS = 1000 bytes, so the segments start at 1001, 2001, 3001, ...*

As in any pipelined RDT, the sender keeps a **send window**.

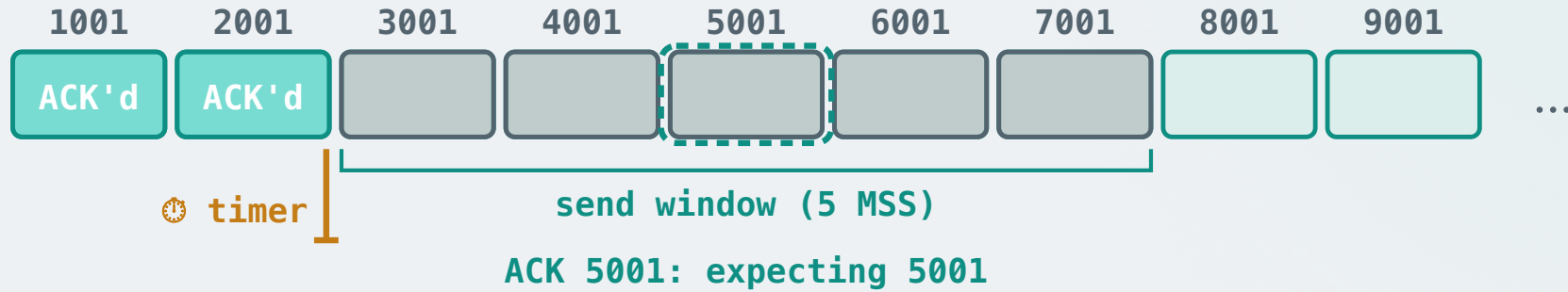
*e.g. a window of 5 MSS, i.e. 5 segments.*

Suppose the first two segments have already been acknowledged, so the window base starts at the third segment, seq 3001.

The sender may send anything within the window, so segments 3001–7001 go out.

Then start **one timer**, if none is running.

# The TCP sender rules: overview



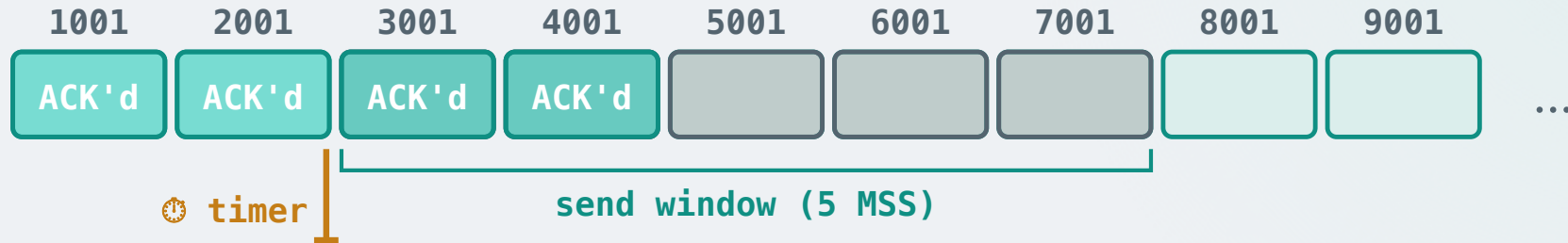
## ACK received

A cumulative ACK slides the window forward.

Restart the timer if unacknowledged data remains; stop it if everything is confirmed.

*e.g. the sender receives ACK 5001: the receiver is expecting the segment starting at 5001.*

# The TCP sender rules: overview



## ACK received

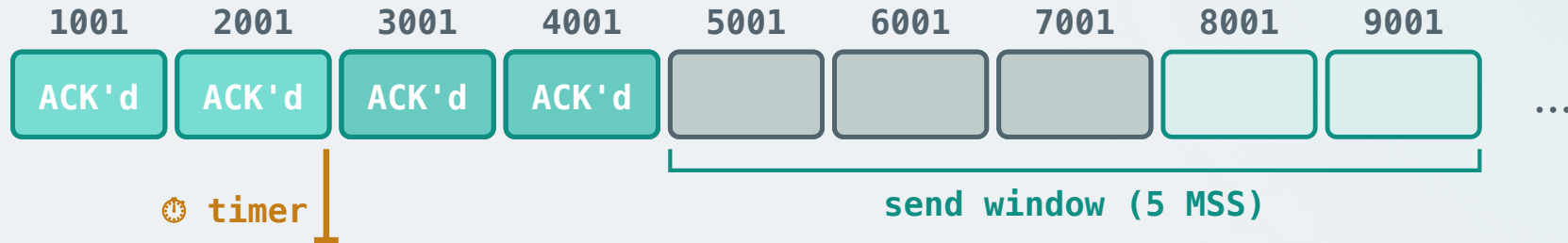
A cumulative ACK slides the window forward.

Restart the timer if unacknowledged data remains; stop it if everything is confirmed.

*e.g. the sender receives ACK 5001: the receiver is expecting the segment starting at 5001.*

*Since ACKs are cumulative, segs 3001 and 4001 are both acknowledged.*

# The TCP sender rules: overview



## ACK received

A cumulative ACK slides the window forward.

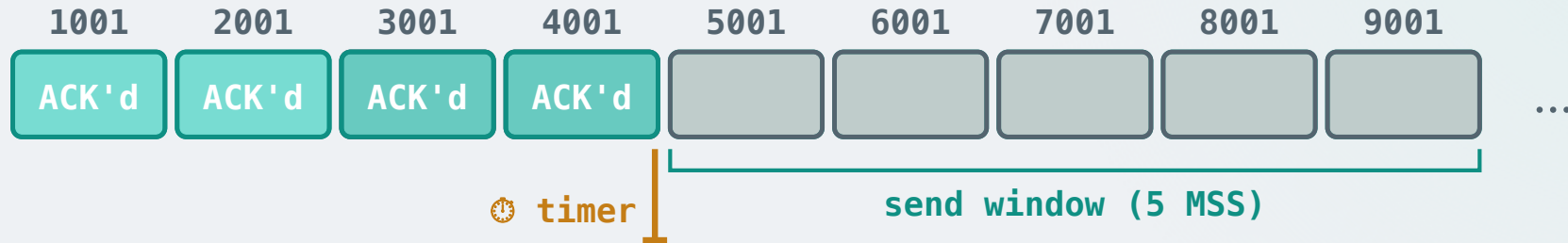
Restart the timer if unacknowledged data remains; stop it if everything is confirmed.

*e.g. the sender receives ACK 5001: the receiver is expecting the segment starting at 5001.*

*Since ACKs are cumulative, segs 3001 and 4001 are both acknowledged.*

*So the window base moves to 5001, and the window now covers 5001–9001.*

# The TCP sender rules: overview



## ACK received

A cumulative ACK slides the window forward.

Restart the timer if unacknowledged data remains; stop it if everything is confirmed.

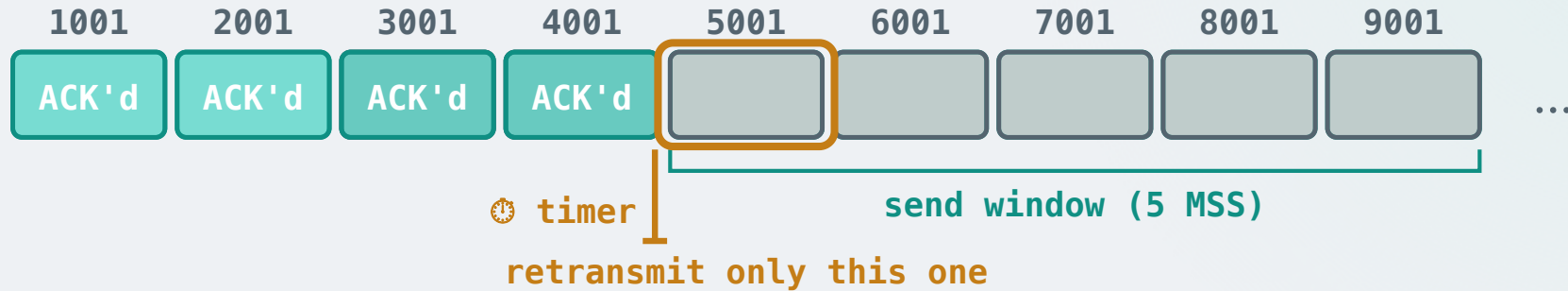
*e.g. the sender receives ACK 5001: the receiver is expecting the segment starting at 5001.*

*Since ACKs are cumulative, segs 3001 and 4001 are both acknowledged.*

*So the window base moves to 5001, and the window now covers 5001–9001.*

*Segs 8001 and 9001 can now go out, and the timer restarts on 5001.*

# The TCP sender rules: overview



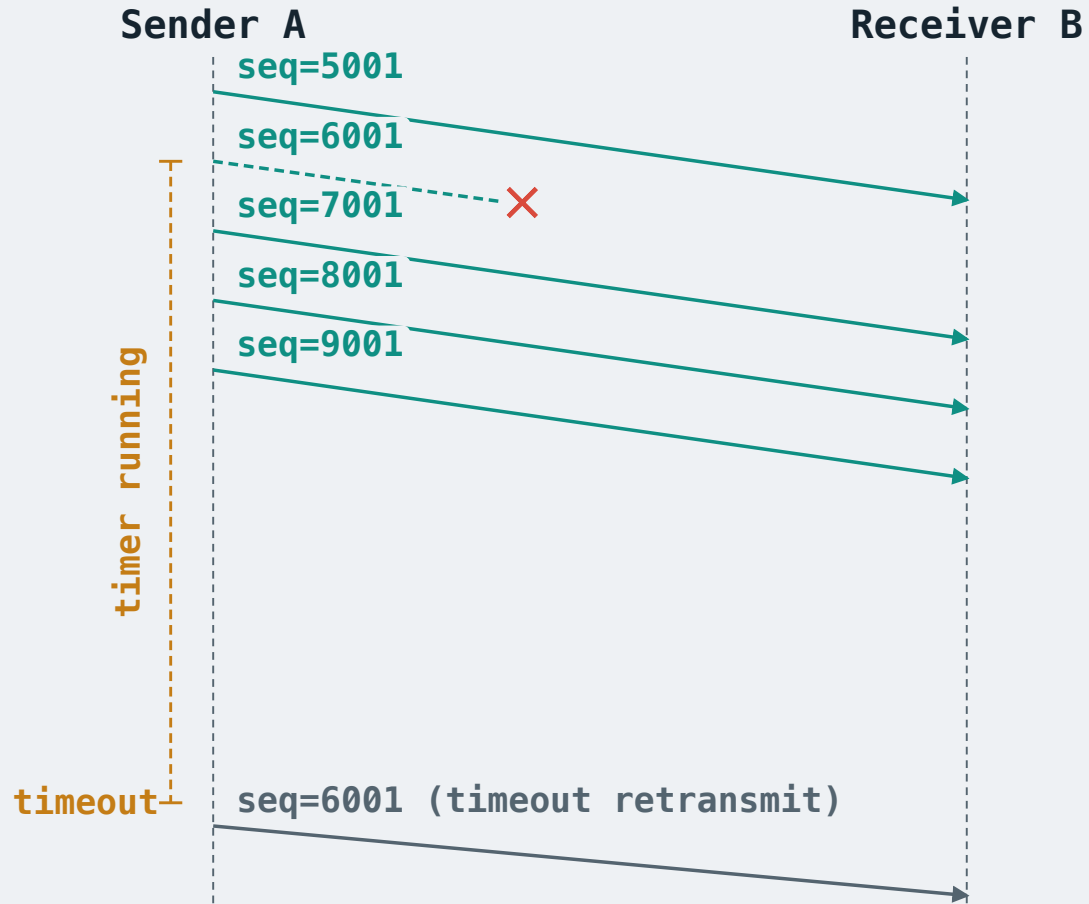
## Timeout

Retransmit only the **oldest unacknowledged segment**, then restart the timer.

This is different from **Go-Back-N**. Recall that in GBN, the sender retransmits every in-flight packet in the window; TCP retransmits only *one*.

*e.g. here GBN would retransmit all five segs, 5001–9001; TCP retransmits only seg 5001.*

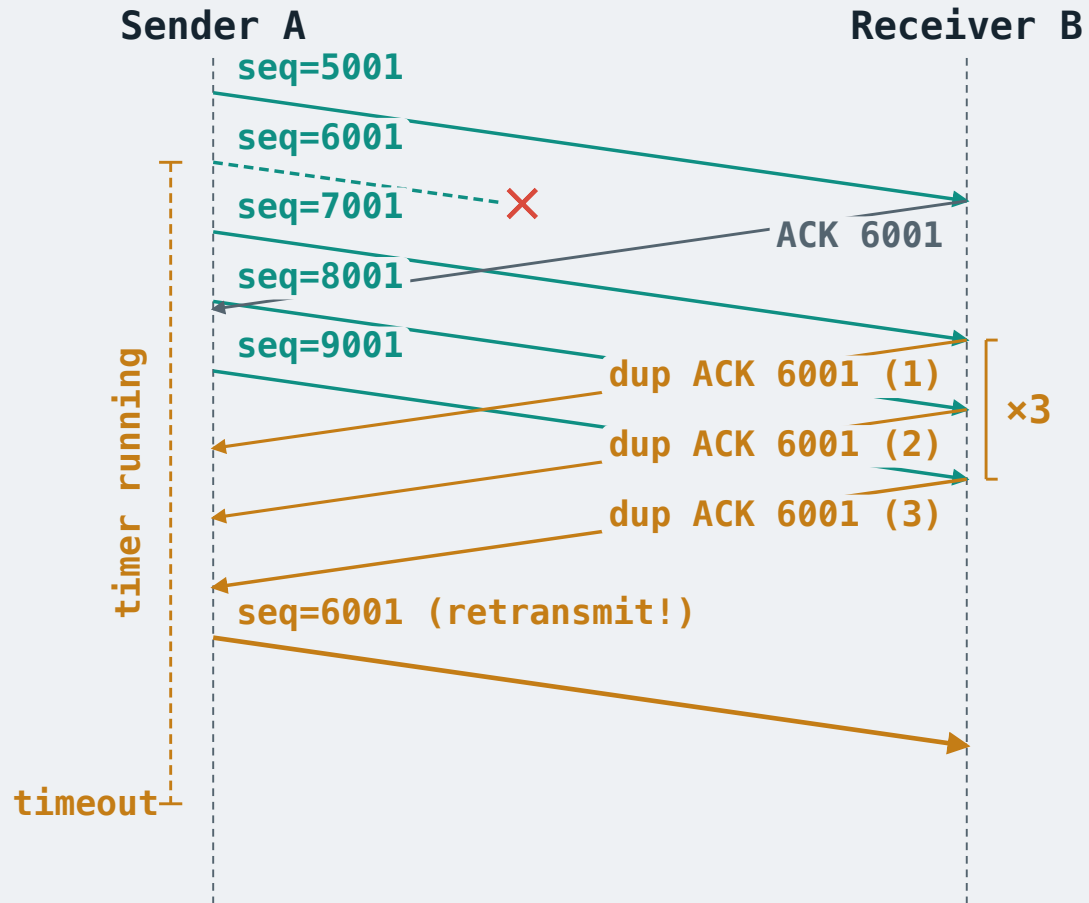
# TCP fast retransmit



## The problem

Timeout period is relatively long. Suppose seg 6001 is lost — in Go-Back-N, the sender must wait until timeout before it can retransmit. This can take a long time.

# TCP fast retransmit



## The problem

Timeout period is relatively long. Suppose seg 6001 is lost — in Go-Back-N, the sender must wait until timeout before it can retransmit. This can take a long time.

## Duplicate ACKs — an early signal

The receiver ACKs the last in-order byte. When 7001, 8001, and 9001 arrive but 6001 is missing, each generates another dup ACK 6001.

A stuck ACK (not advancing) means the receiver is still waiting for the same segment — already an *early indication of loss*, before the timer fires.

## Fast retransmit

After **3 duplicate ACKs**, retransmit the missing segment immediately — without waiting for the timeout.

# TCP receiver rules: overview

## Like Go-Back-N: cumulative ACK

The receiver sends a **cumulative ACK** for the highest in-order byte received. If a segment arrives out of order, the receiver sends a **duplicate ACK** — ACKing the next expected sequence number, signalling the gap to the sender.

## Like Selective Repeat: receiver buffer

Unlike Go-Back-N, the receiver **buffers out-of-order segments** within the receive window rather than discarding them. Once the missing segment arrives and the gap is filled, buffered data is delivered to the application in order.

## Further improvements

There are further recommendations for ACK generation (e.g. to avoid sending too many ACKs) in **RFC 1122** and **RFC 2581**.

TCP can also negotiate **selective ACK options** [RFC 2018] — the receiver reports exactly which byte ranges it holds, letting the sender retransmit only the missing pieces, much like Selective Repeat.

# TCP timeout: the challenge

## The timer should track RTT

The sender's retransmit timer is set to roughly the **round-trip time** — just long enough for a packet to receive its ACK. Too short → spurious retransmissions waste bandwidth. Too long → slow recovery when a segment really is lost.

## The problem: RTT is not a constant number in the real world

Network load, queueing, and path changes cause RTT to fluctuate — sometimes dramatically.

A fixed timeout value cannot adapt.

This is why TCP **estimates RTT dynamically**, in real time, and updates its RTT estimate continuously.

# TCP RTT estimation

## SampleRTT

EstimatedRTT is derived by taking many individual RTT samples.

Each sample is measured from when a segment is transmitted until its ACK is received.

Retransmitted segments are **excluded**: the ACK could be for the original send or the retransmit — the sender cannot tell, so the measurement would be unreliable.

# TCP RTT estimation

## SampleRTT

### EstimatedRTT — exponential weighted moving average

SampleRTT varies noticeably from measurement to measurement. So how does TCP smooth out these variations and derive EstimatedRTT from the samples?

TCP uses a smoothing technique that computes a smooth, adaptive estimate from the samples.

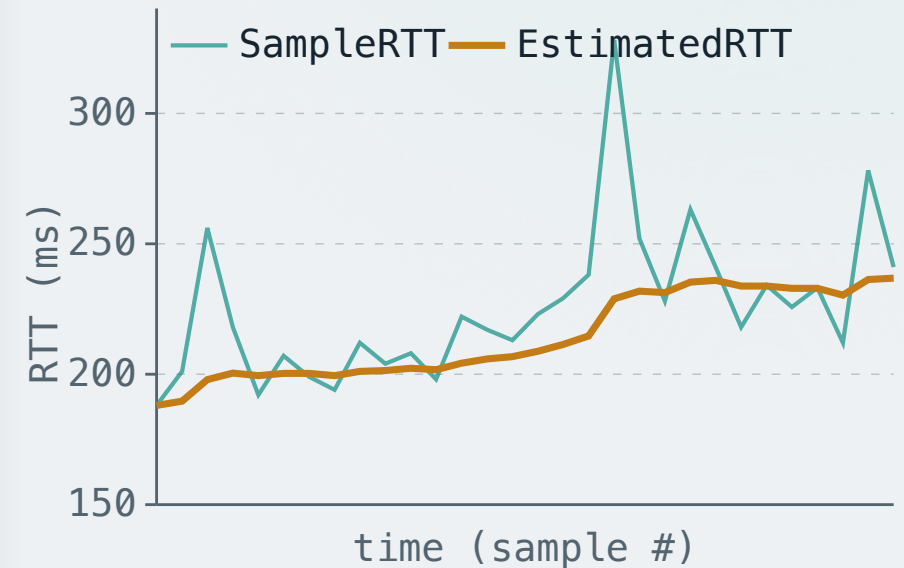
$$\text{EstimatedRTT} = (1-\alpha) \cdot \text{EstimatedRTT} + \alpha \cdot \text{SampleRTT}$$

Typical value:  $\alpha = 0.125$ . Unrolling this one step at a time:

$$\begin{aligned} \text{EstimatedRTT} = & \alpha \cdot \text{Sample}_n + \alpha(1-\alpha) \cdot \text{Sample}_{n-1} + \\ & \alpha(1-\alpha)^2 \cdot \text{Sample}_{n-2} + \dots \end{aligned}$$

Each step further back adds another factor of  $(1-\alpha) = 0.875$  — so older samples count exponentially less.

This is why it is known as an **exponential weighted moving average**: it smooths out SampleRTT to produce EstimatedRTT.



# TCP timeout interval

You might be tempted to simply set the timeout interval to EstimatedRTT.

In practice, this turns out to be too aggressive: due to exponential averaging, EstimatedRTT takes some time to adapt to the real-time RTT.

So when the real RTT suddenly spikes — say the network becomes congested — EstimatedRTT has not caught up yet, and the timer fires too early.

We therefore need a **safety margin** for RTT variability. How do we choose it?

## DevRTT — tracking variability

We also maintain **DevRTT** — a tracker for how much SampleRTT deviates from EstimatedRTT.

$$\text{DevRTT} = (1-\beta) \cdot \text{DevRTT} + \beta \cdot |\text{SampleRTT} - \text{EstimatedRTT}|$$

Typical value:  $\beta = 0.25$ .

## TimeoutInterval

This is how TCP sets the timeout interval, with DevRTT providing the extra safety margin:

$$\text{TimeoutInterval} = \text{EstimatedRTT} + 4 \cdot \text{DevRTT}$$

Empirically, this prevents the timer from firing too early in the case of a sudden spike in RTT.

MATCHING THE SENDER TO THE RECEIVER AND THE NETWORK

Concept 3 of 3

# TCP Flow Control and Congestion Control

---

# Flow control vs congestion control

What are flow control and congestion control? Consider a sender transmitting data to a receiver. The router in the middle stands for the network in between.

Ideally, the sender would send data as fast as it likes. In practice it cannot: there are **two bottlenecks** that could occur.

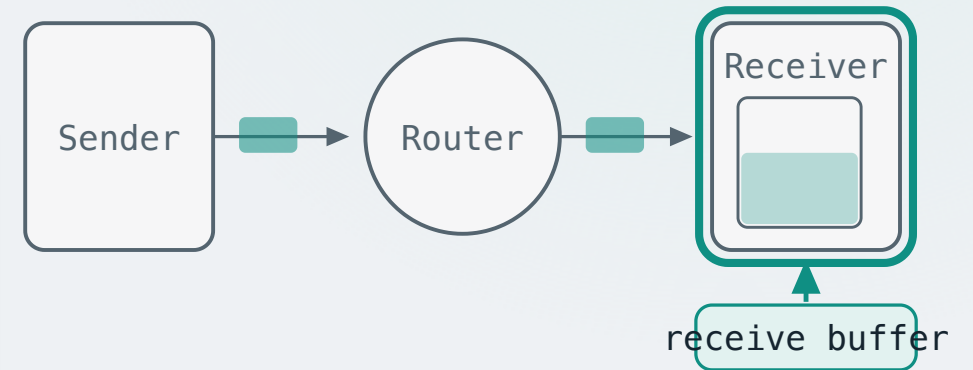
## Flow control

In our previous discussions, we always assumed that the application consumes data as soon as it arrives at the receiver.

In reality, the application may read slowly, so the receiver must hold incoming data and feed it to the application gradually. This is why the receiver maintains a **receive buffer**.

This buffer has a limited size: if the sender transmits too fast, it overflows and segments are lost.

Flow control prevents this by matching the sender's transmission rate to the speed at which the receiver's application reads, so the buffer does not overflow.



# Flow control vs congestion control

What are flow control and congestion control? Consider a sender transmitting data to a receiver. The router in the middle stands for the network in between.

Ideally, the sender would send data as fast as it likes. In practice it cannot: there are **two bottlenecks** that could occur.

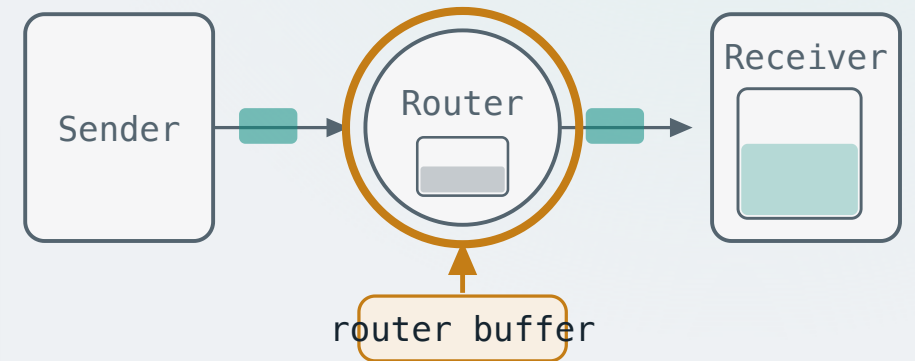
## Congestion control

We have already seen that datagrams travel through the Internet across a series of routers.

A router takes time to process each incoming datagram and cannot always forward it immediately, so it keeps a **router buffer**.

If too many senders transmit simultaneously, the buffer of some router along the route fills up and segments are dropped: the network is **congested**.

Congestion control prevents this by limiting each sender's rate to match what the network can carry.



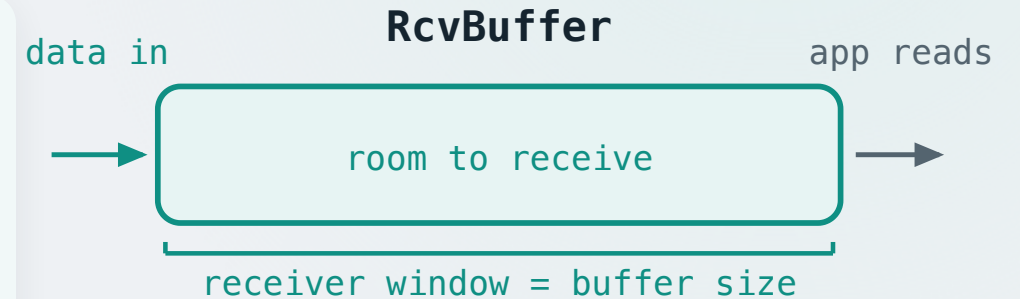
# TCP flow control: the receive window

## The receive buffer and floating window

The receiver allocates a fixed-size receive buffer (**RcvBuffer**) for the connection. Incoming data sits there until the application reads it.

Recall that in RDT, the **receiver window** is the range of packets the receiver is willing to buffer.

In our previous discussions, we assumed the application consumes data as soon as it arrives. The buffer was therefore always fully available to receive, so we always set **receiver window = buffer size**.



# TCP flow control: the receive window

## The receive buffer and floating window

The receiver allocates a fixed-size receive buffer (**RcvBuffer**) for the connection. Incoming data sits there until the application reads it.

Recall that in RDT, the **receiver window** is the range of packets the receiver is willing to buffer.

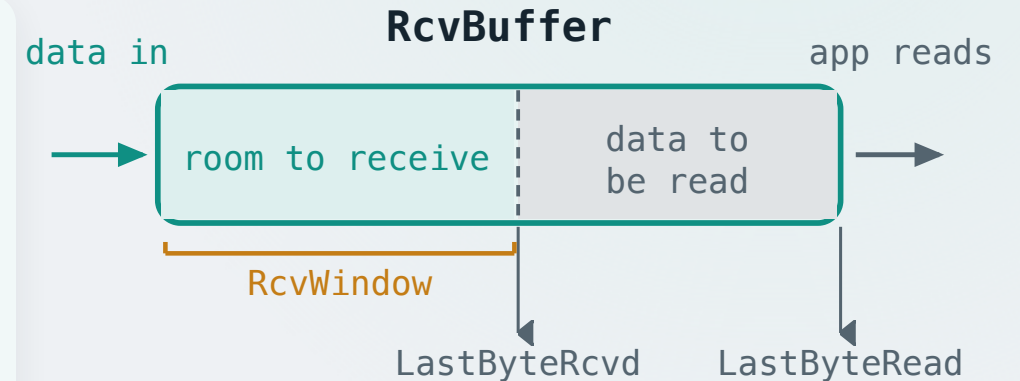
In our previous discussions, we assumed the application consumes data as soon as it arrives. The buffer was therefore always fully available to receive, so we always set **receiver window = buffer size**.

In TCP, the application reads at its own pace — so the buffer may contain unread data that reduces available space.

The unread portion is then  $[\text{LastByteRcvd} - \text{LastByteRead}]$ ; thus the receiver window **floats**:

$$\text{RcvWindow} = \text{RcvBuffer} - [\text{LastByteRcvd} - \text{LastByteRead}]$$

e.g.  $\text{RcvBuffer} = 4000$  bytes,  $\text{LastByteRead} = 2000$ ,  $\text{LastByteRcvd} = 3500 \rightarrow$   
 $\text{RcvWindow} = 4000 - (3500 - 2000) = \mathbf{2500}$  bytes.



**LastByteRcvd**: the sequence number of the last byte received.

**LastByteRead**: the sequence number of the last byte the application has read.

# Advertising and enforcing the receive window

## Advertising the window

How does the TCP receiver advertise its current window to the sender?

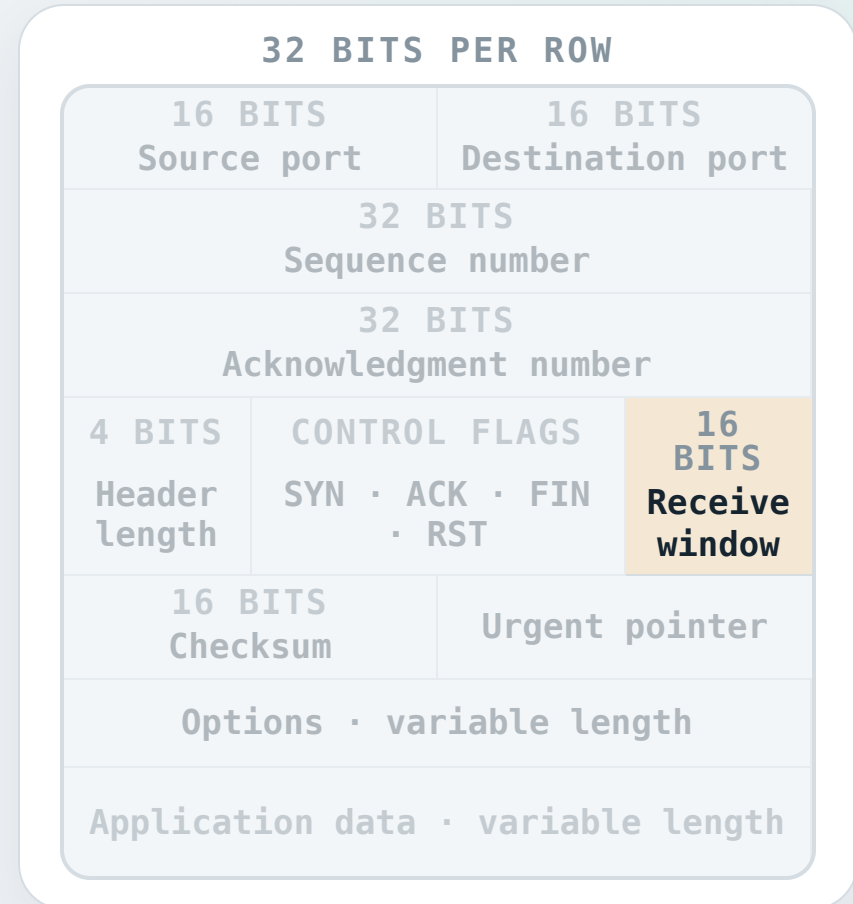
In TCP, the receiver piggybacks RcvWindow on every segment it sends, carried in the **Receive Window** field of the TCP header.

## The TCP sender's rule

**LastByteSent – LastByteAcked ≤ RcvWindow**

Recall from our earlier discussion: LastByteAcked is essentially the window **base**, and LastByteSent is essentially **nextseqnum**. So the left side counts the bytes in [base, nextseqnum) — the data in flight.

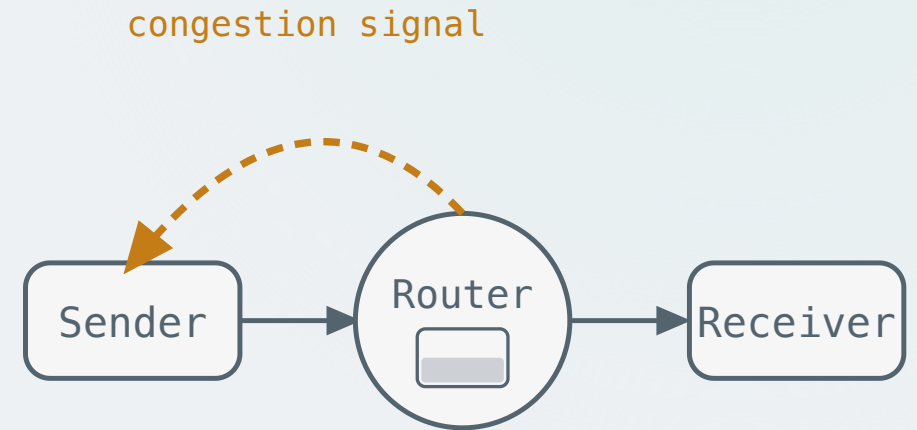
In other words, the TCP sender ensures the amount of data in flight never exceeds the advertised RcvWindow.



# Approaches to congestion control

## Network-assisted

Routers explicitly notify senders to slow down — for example, by setting a congestion bit in a packet header, or stating the supported rate. Requires router cooperation throughout the path.



# Approaches to congestion control

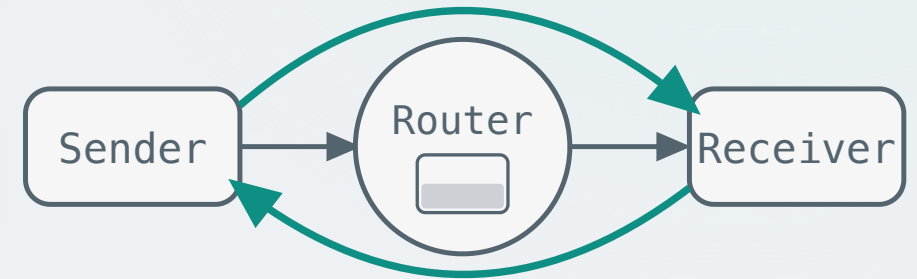
## Network-assisted

Routers explicitly notify senders to slow down — for example, by setting a congestion bit in a packet header, or stating the supported rate. Requires router cooperation throughout the path.

## End-to-end

No explicit feedback from the network. The sender infers congestion from observable signals — primarily **packet loss** and increased **delay**.

TCP uses the **end-to-end** approach — congestion is inferred from **loss events** in the network.



# TCP congestion detection and action: an overview

## Detection

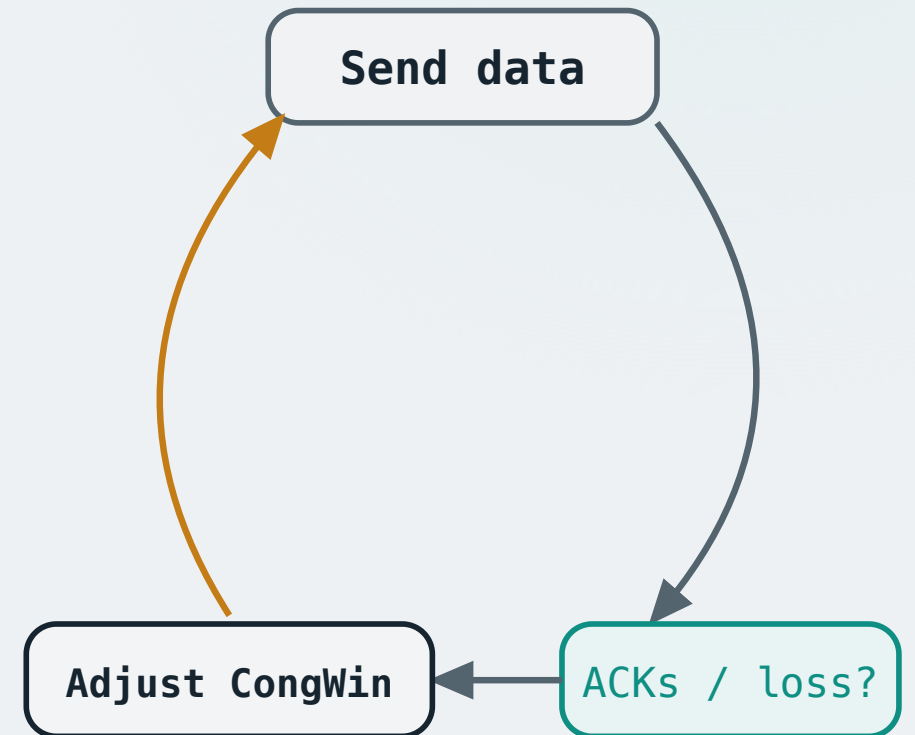
**3 duplicate ACKs** — one segment lost, but others still arriving (mild congestion).

**Timeout** — **nothing** gets through; the network appears unresponsive (serious congestion).

## Action

The sender maintains a **congestion window (CongWin)** — a cap on how much unacknowledged data it may have in flight at any time.

On detecting congestion, **shrink** CongWin.



# How TCP matches its sending rate

## The in-flight constraint

$$\text{LastByteSent} - \text{LastByteAcked} \leq \text{CongWin}$$

The sender always keeps its unacknowledged data in flight below this limit.

## Per-RTT implication

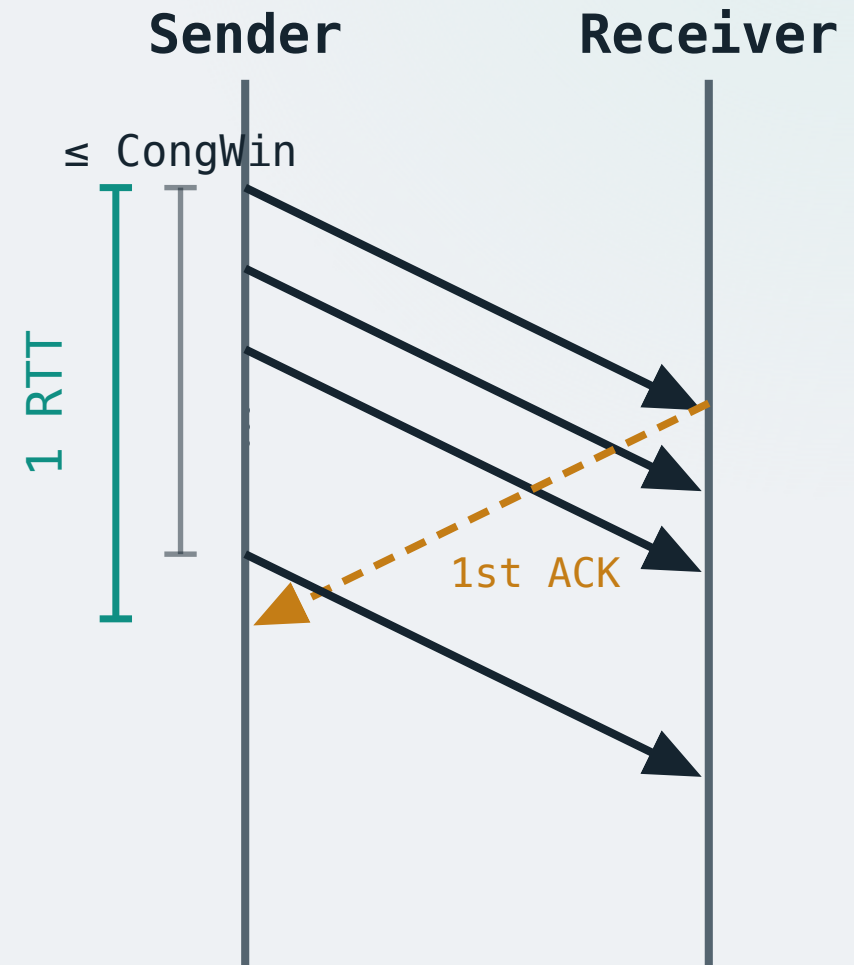
The sender transmits segments back-to-back — all remain in flight until the first ACK returns, which takes one RTT.

So within one RTT, everything sent is simultaneously in flight. Since the constraint says at most CongWin bytes can be in flight, **the sender can send at most CongWin bytes per RTT.**

## The corresponding sending rate

Since TCP carries a continuous byte stream, the corresponding sending rate is:

$$\text{sending rate} \approx \text{CongWin} / \text{RTT} \quad \text{bytes/sec}$$



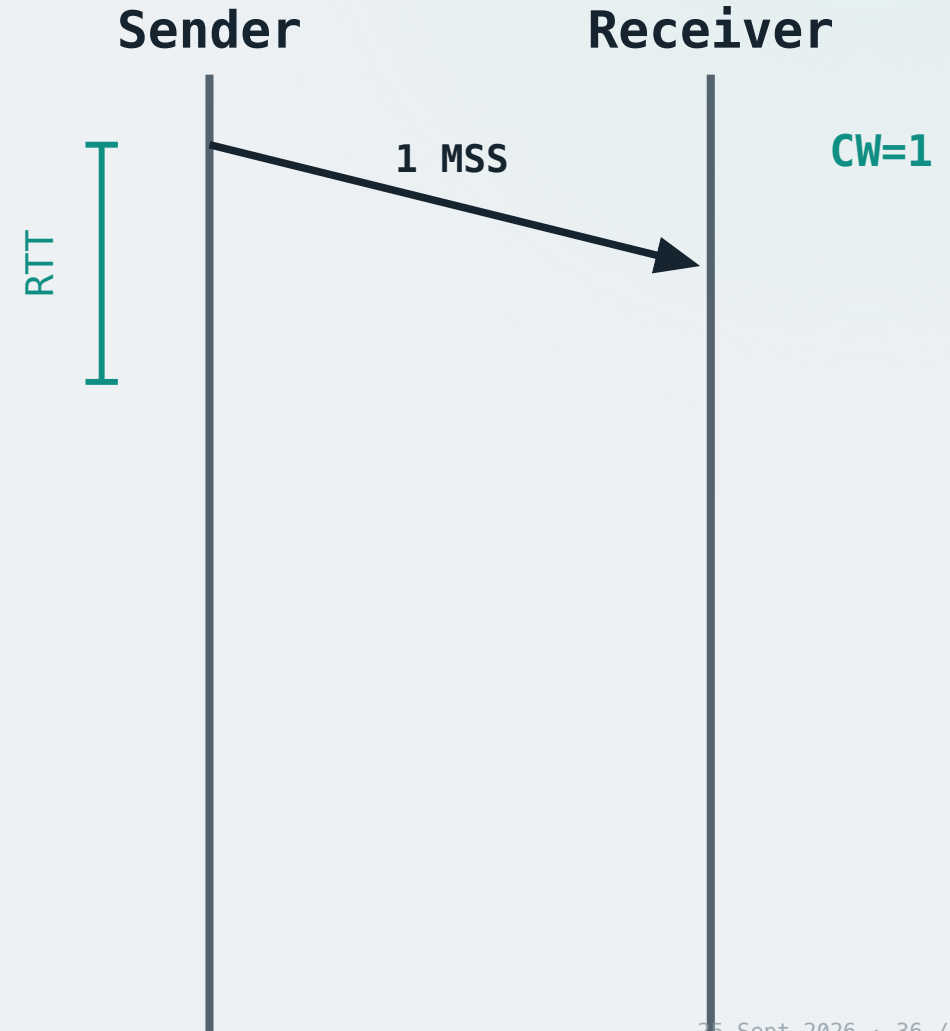
# TCP • slow start

## Starting conservatively

When a connection begins, the sender knows nothing about the path — start conservatively: **CongWin = 1 MSS**.

Since CongWin = 1 MSS, the in-flight constraint means the sender may only send **1 MSS per RTT**.

Each segment carries exactly 1 MSS of data. Send one segment, wait for its ACK.



# TCP • slow start

## Growing the window

Each ACK confirms the network handled the segment — likely no congestion yet.

### **SLOW-START RULE**

**For every ACK received: CongWin += 1 MSS**

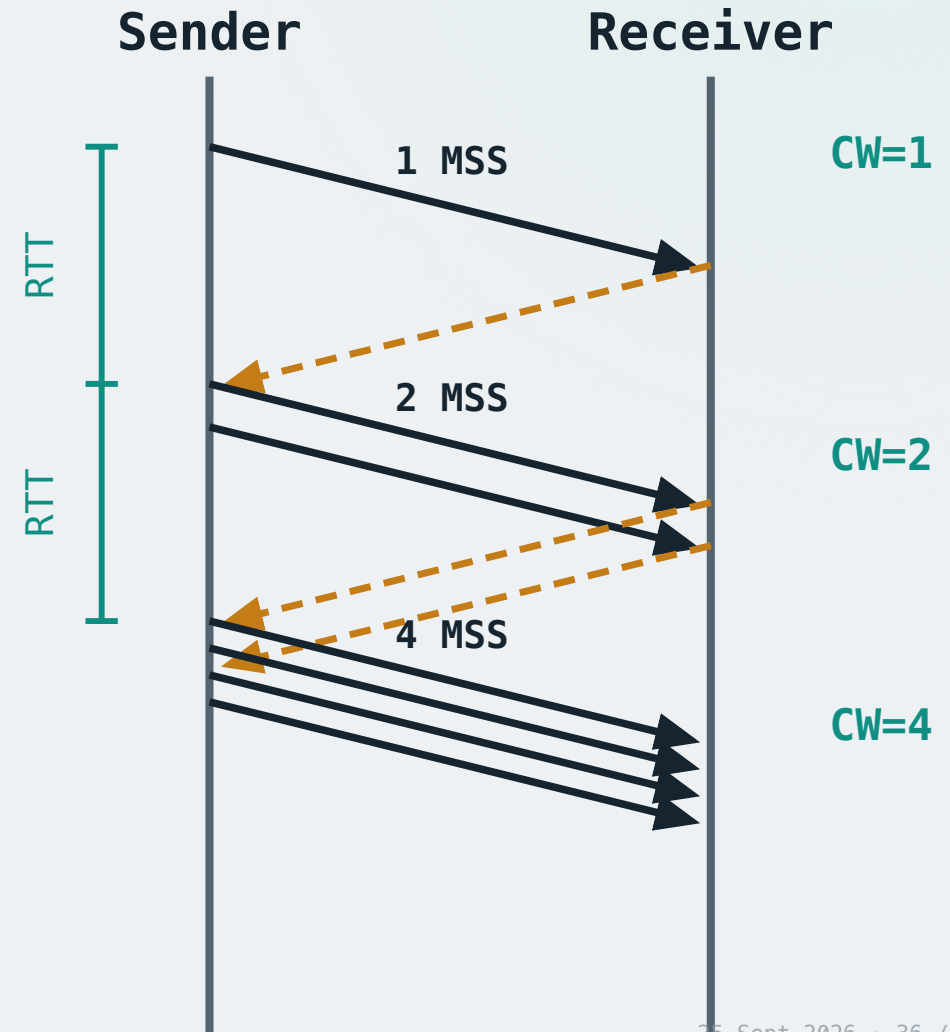
CongWin increases to 2 — in the next RTT the sender can send **2 MSS**.

Two more ACKs received — each adds 1 MSS. CongWin increases to 4, and in the next RTT we can send **4 MSS**.

The pattern: the growth rate depends on the current window size.

### **SLOW-START EMPIRICAL RULE**

**CongWin doubles every RTT – 1, 2, 4, 8, 16... – exponential growth**

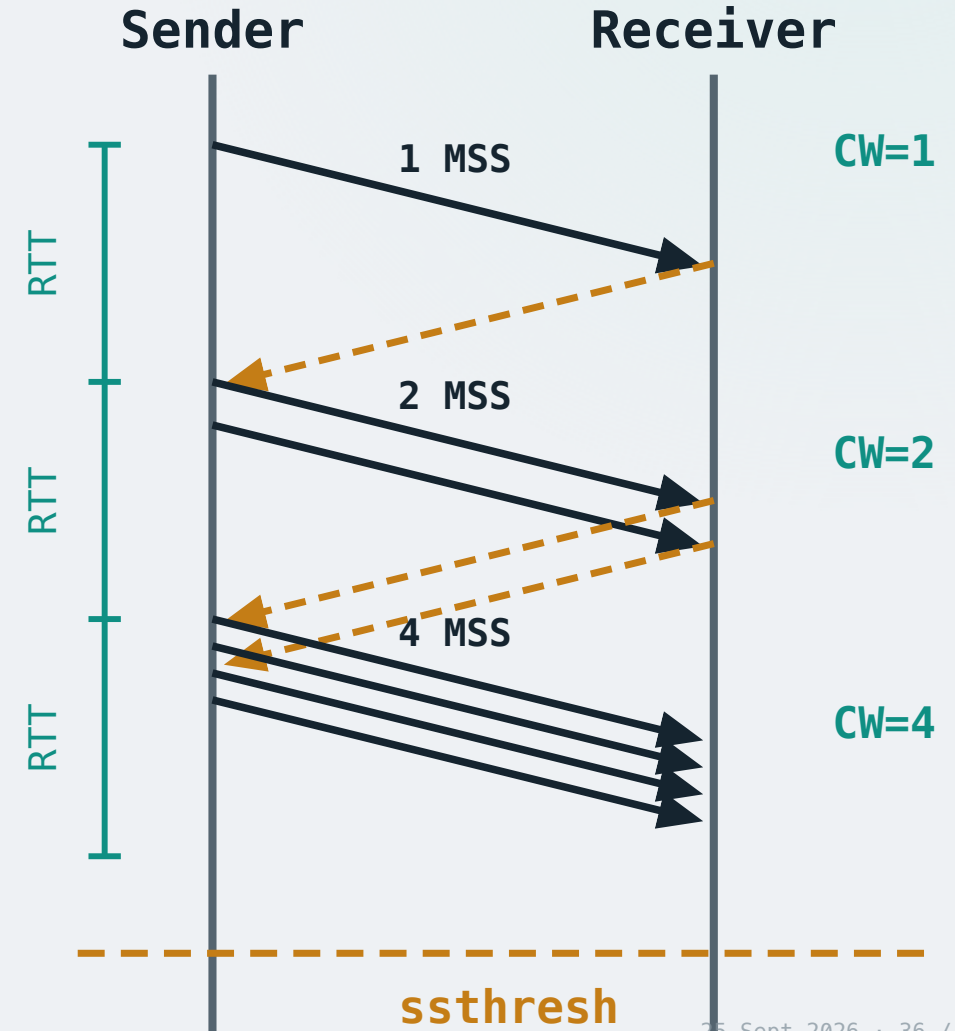


# TCP • slow start

## The threshold

Exponential growth can't last forever. A threshold called **ssthresh** (slow-start threshold) marks where slow start ends.

When  $\text{CongWin} \geq \text{ssthresh}$  → switch to **congestion avoidance** (next frame).



# TCP · congestion avoidance

Suppose **ssthresh** = 4 MSS — the point where congestion previously occurred.

Sender

Receiver

CW=4

# TCP • congestion avoidance

## Additive increase

Once  $\text{CongWin} \geq \text{ssthresh}$ , growth switches from exponential to linear.

### ADDITIVE INCREASE

If no loss detected:  $\text{CongWin} += 1 \text{ MSS per RTT}$

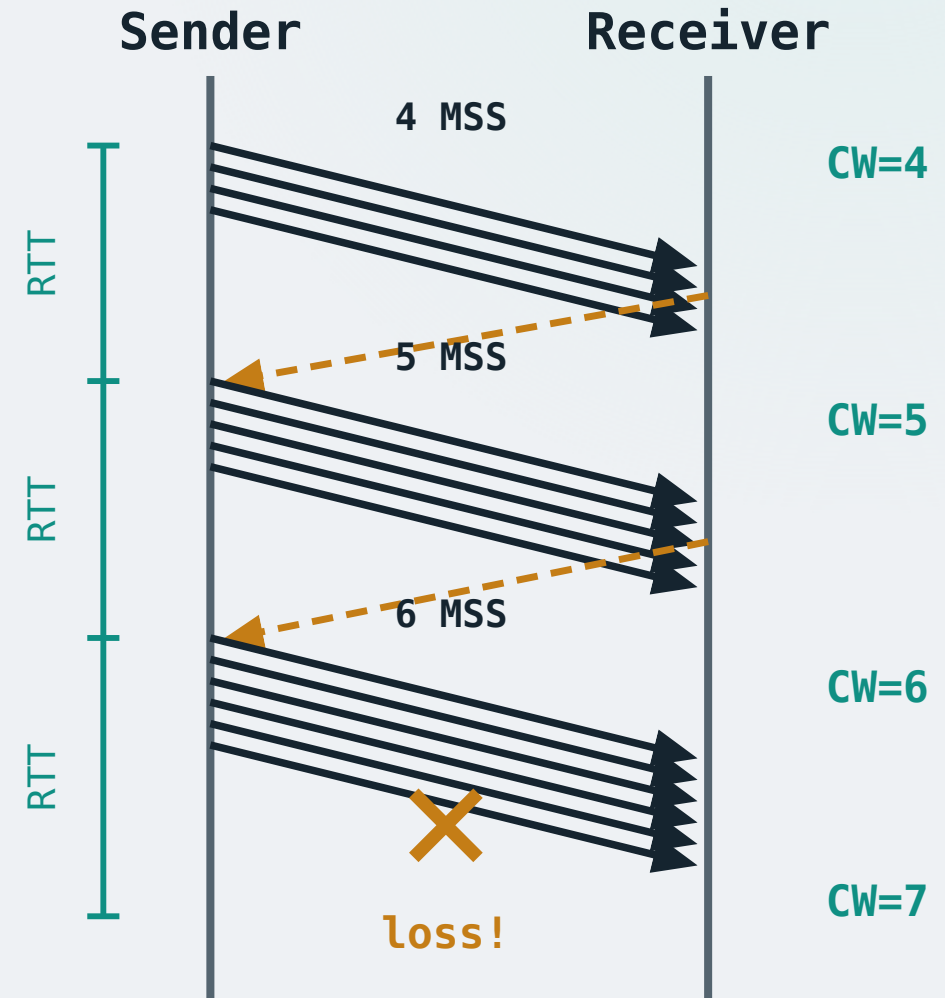
## Multiplicative decrease

We keep additive increase until we hit a loss event (next couple of frames), where we cut  $\text{CongWin}$  in half.

### MULTIPLICATIVE DECREASE

Upon loss event:  $\text{CongWin} = \text{CongWin} / 2$

Together, this is known as the **AIMD** rule — Additive Increase, Multiplicative Decrease.



# Why AIMD?

## Multiplicative Increase, Additive Decrease (MIAD)

Multiplicative increase overshoots capacity aggressively, while additive decrease backs off too slowly — congestion persists.

## Additive Increase, Additive Decrease (AIAD)

AIAD controls congestion correctly for a single flow, but it fails to address a different issue: **fairness**. The goal of fairness is that every flow sharing a bottleneck link should converge to an equal share of bandwidth.

Suppose two flows share a 10 Mbps link: Flow A = 8 Mbps, Flow B = 2 Mbps. This is not fair — a good congestion control design should correct the imbalance while also preventing congestion.

Additive increase: both add 1 Mbps → A = 9 Mbps, B = 3 Mbps. Total exceeds capacity → loss.

Additive decrease: both subtract 1 Mbps → A = 8 Mbps, B = 2 Mbps. Back to the start — the unfairness never corrects.

**AIAD does not solve the fairness problem.**

## Additive Increase, Multiplicative Decrease (AIMD)

Additive increase probes gently; multiplicative decrease reacts sharply. Converges to both **efficiency** and **fairness**.

*Take-home question: using the same example (A = 8 Mbps, B = 2 Mbps), why does multiplicative decrease close the gap where additive decrease could not?*

# TCP • reacting to loss

Cutting CongWin in half is not the whole story. What TCP really adjusts on loss is the **slow-start threshold** — our empirical estimate of the network's capacity.

## ON ANY LOSS EVENT

**ssthresh** = CongWin / 2

### Case 1: Timeout

Set ssthresh = CongWin / 2.

Reset **CongWin = 1 MSS** and enter **slow start**.

A timeout is a very serious signal of severe congestion — be conservative and restart from scratch.

### Case 2: Three duplicate ACKs

Set ssthresh = CongWin / 2, then enter **fast recovery** (next frame).

*This is the TCP Reno design. In TCP Tahoe, there is no distinction — both cases reset CongWin = 1 MSS and enter slow start.*

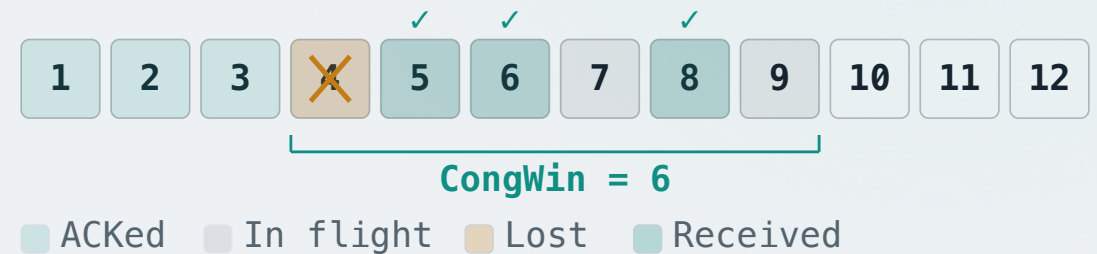
# TCP • fast recovery

Suppose CongWin = 6. Segments 1–3 have been acknowledged, and the sender has more data to transmit.

The sender fills the congestion window: segments 4–9 are now in flight.

The sender then receives three dup ACKs for segment 4. What does this mean?

Segment 4 is likely **lost** during transmission. But importantly, three ACKs means three of the in-flight packets 5–9 reached the receiver — suppose those are segments 5, 6, 8.

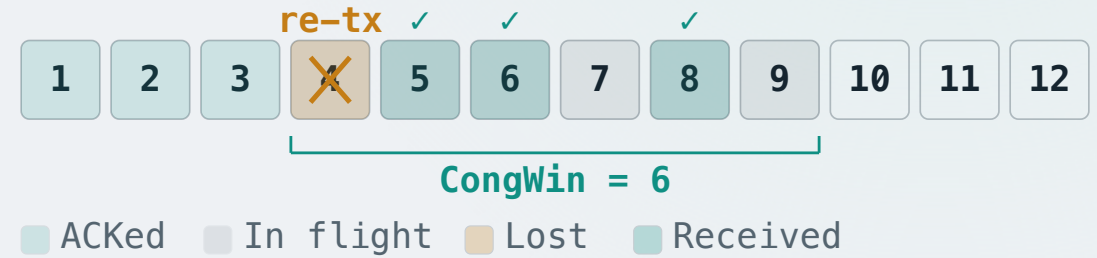


# TCP • fast recovery

What should the sender do now?

**Fast retransmit:** immediately resend segment 4.

The sender enters **fast recovery**.



# TCP • fast recovery

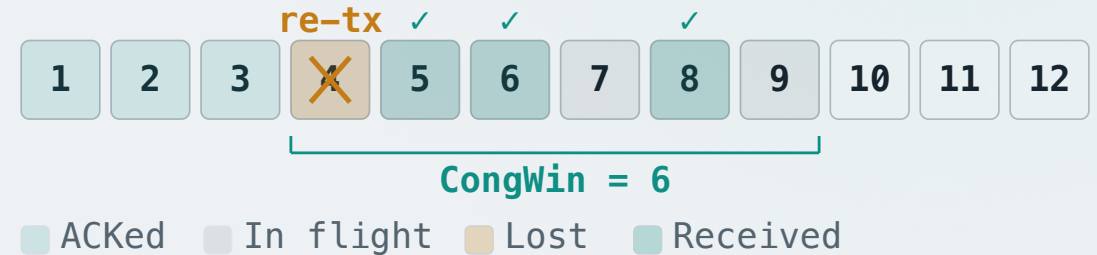
## FAST RECOVERY – ENTER

$\text{ssthresh} = \text{CongWin} / 2 = 3$

$\text{CongWin} = \text{ssthresh} + 3 = 6$

### Why inflate by 3?

- CongWin limits how many segments can be in flight. If we simply apply AIMD and set  $\text{CongWin} = \text{ssthresh} = 6/2 = 3$ , the window starts at base 4 and covers segments 4, 5, 6.
- However, by observing 3 dup ACKs, we know 3 segments among 5–9 are no longer in flight. Segments **5, 6, 8** are confirmed received and do not occupy in-flight slots.
- The only truly in-flight segments are **4, 7, 9**. This justifies inflating CongWin to  $6 = \text{ssthresh} (3) + \text{confirmed received} (3)$ , reflecting that only 3 segments are truly in flight.

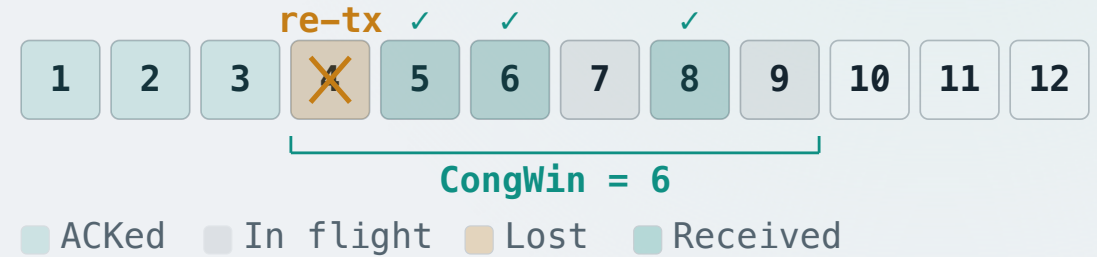


# TCP · fast recovery

## FAST RECOVERY – PER DUP ACK

**For each additional dup ACK: CongWin += 1 MSS**

For the same reason, each additional dup ACK signifies one less segment is truly in flight — the window can grow by 1.



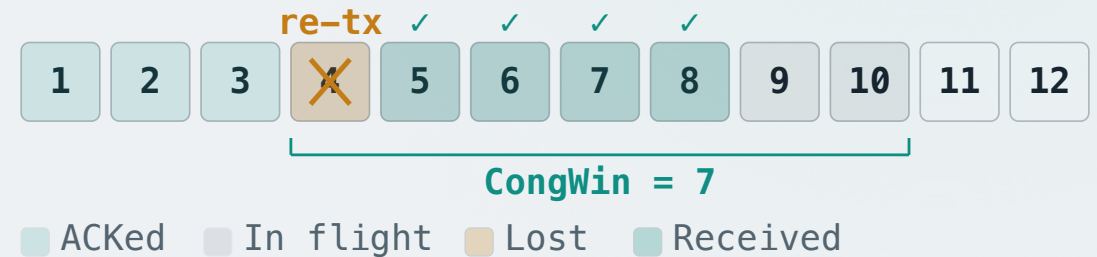
# TCP • fast recovery

## FAST RECOVERY – PER DUP ACK

For each additional dup ACK:  $\text{CongWin} += 1 \text{ MSS}$

For the same reason, each additional dup ACK signifies one less segment is truly in flight — the window can grow by 1.

Segment 7 received, yet segment 4 still pending (another dup ACK for 4): **CongWin = 7** — can now send segment 10.



# TCP • fast recovery

## FAST RECOVERY – PER DUP ACK

For each additional dup ACK:  $\text{CongWin} += 1 \text{ MSS}$

For the same reason, each additional dup ACK signifies one less segment is truly in flight — the window can grow by 1.

Segment 7 received, yet segment 4 still pending (another dup ACK for 4): **CongWin = 7** — can now send segment 10.

Segment 9 received, yet segment 4 still pending (another dup ACK for 4): **CongWin = 8** — can now send segment 11.



# TCP · fast recovery exit

Now suppose the retransmitted segment 4 has been received, so the receiver sends a cumulative ACK for everything through segment 9 — it sends ACK 10.



# TCP • fast recovery exit

Now suppose the retransmitted segment 4 has been received, so the receiver sends a cumulative ACK for everything through segment 9 — it sends ACK 10.

## FAST RECOVERY – EXIT RULE

On new ACK:

- Set **CongWin** = **ssthresh**
- Enter **congestion avoidance**

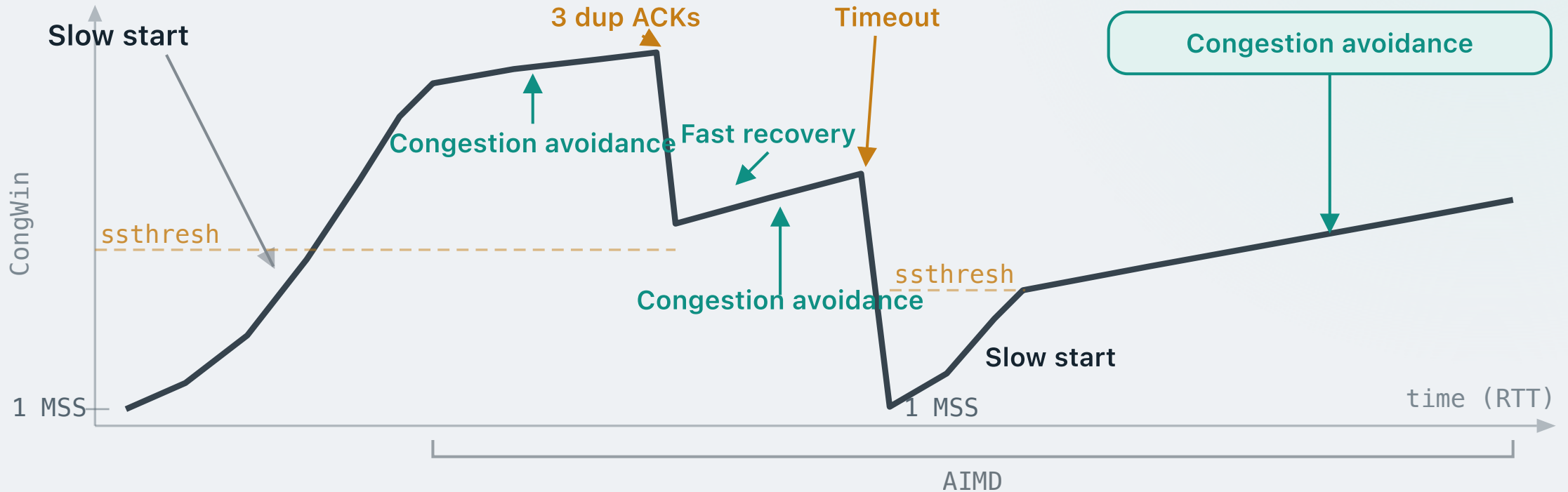
The sender receives ACK 10. Base advances to segment 10.

**CongWin** = **ssthresh** = 3 — the inflated window deflates back to the halved value.

Enter **congestion avoidance**. Fast recovery is complete.



# TCP · congestion control overview



# Congestion control — summary

TCP begins in **slow start**, doubling CongWin each RTT until ssthresh, then grows linearly in **congestion avoidance**.

Three dup ACKs trigger **fast recovery** — ssthresh and CongWin halve. A **timeout** drops CongWin to 1 MSS and restarts slow start.

The resulting **AIMD sawtooth** continuously probes for available bandwidth while backing off sharply on congestion signals.

# Acknowledgements

Lecture notes are developed based on slides from the following sources:

- Dr Mengmeng Ge's lecture notes for COSC264, University of Canterbury, 2024
- Assoc Prof Dan Kim's lecture notes for COSC264, University of Canterbury, 2017
- Prof Aleksandar Kuzmanovic's lecture notes for CS340, Northwestern University, 2005 — [https://users.cs.northwestern.edu/~akuzma/classes/CS340-w05/lecture\\_notes.htm](https://users.cs.northwestern.edu/~akuzma/classes/CS340-w05/lecture_notes.htm)
- Dr Barry Wu's lecture notes for COSC264, University of Canterbury

# References

James F. Kurose & Keith W. Ross, *Computer Networking: A Top-Down Approach Featuring the Internet*, Addison Wesley, 3rd edition, 2005.

Chapter 3 Transport Layer (3.1 – 3.3, 3.5 – 3.7)

William Stallings, *Data and Computer Communications*, Pearson, 10th edition, 2013.

Chapter 15 Transport Protocols

Andrew S. Tanenbaum & David J. Wetherall, *Computer Networks*, Prentice Hall, 5th edition, 2010.

Chapter 6 The Transport Layer (6.2, 6.3, 6.5)